

Preparando o ambiente

Trabalhando com Texto em Python I · Unidade 0

CiberExt 26-29 · FEELT38103 · Universidade Federal de Uberlândia

2026

Sumário

1. Python 3	1
2. Ambiente virtual (venv)	1
3. Instalar as bibliotecas	2
4. Baixar o modelo de linguagem do spaCy	2
5. Os dados (corpus)	2
6. Rodar os exemplos	3
7. Rodar os exercícios (correção automática)	3
Checklist final	3

Objetivos

Antes de começar as unidades, você precisa deixar o computador pronto. Ao final desta preparação, você terá:

- **Python 3** instalado e funcionando;
- um **ambiente virtual** isolado para o curso;
- as **bibliotecas** (spaCy, pandas, scikit-learn, matplotlib, Jupyter);
- o **modelo de linguagem** do spaCy baixado;
- os **dados** (corpus) na pasta certa;
- os **exercícios** rodando com correção automática.

Dica: faça esta preparação **uma vez**. Depois, sempre que for estudar, basta **ativar o ambiente virtual** (passo 2) e começar.

1. Python 3

O curso usa **Python 3.9 ou mais novo**. Verifique se já tem, no terminal:

```
python3 --version
```

Se aparecer algo como **Python 3.11.x**, está pronto. Se não tiver Python, baixe em python.org/downloads (Windows/macOS) ou instale pelo gerenciador do seu sistema (no macOS, via [Homebrew](https://brew.sh/): `brew install python`).

2. Ambiente virtual (venv)

Um **ambiente virtual** isola as bibliotecas do curso, sem bagunçar o resto do sistema. Crie um na pasta do curso:

```
python3 -m venv .venv
```

E **ative-o** (você repete este passo toda vez que for estudar):

```
# macOS / Linux
source .venv/bin/activate

# Windows (PowerShell)
.venv\Scripts\Activate.ps1
```

Quando ativo, o nome (**.venv**) aparece no início da linha do terminal. Para sair, use **deactivate**.

3. Instalar as bibliotecas

Com o ambiente **ativado**, atualize o **pip** e instale tudo de uma vez:

```
pip install --upgrade pip
pip install -r requirements.txt
```

Se preferir instalar manualmente:

```
pip install spacy pandas scikit-learn matplotlib jupyterlab
```

4. Baixar o modelo de linguagem do spaCy

O spaCy separa a biblioteca do **modelo** treinado. As Unidades 2 e 3 usam o modelo **pequeno** de inglês; a Unidade 5 usa o **médio**:

```
python -m spacy download en_core_web_sm
python -m spacy download en_core_web_md
```

Teste se carregou:

```
python -c "import spacy; nlp = spacy.load('en_core_web_sm'); print('spaCy OK!')"
```

5. Os dados (corpus)

Os exemplos usam alguns arquivos de texto (notícias) e uma planilha de comentários (o corpus **SOCC**). Coloque-os numa pasta chamada **data/**, ao lado dos seus notebooks:

```
data/
  NYT_1991-01-16-A15.txt
  WP_1990-08-10-25A.txt
  WP_1991-01-17-A1B.txt
```

```
socc_gnm_articles.csv
```

Esses arquivos vêm do material do curso de referência *Applied Language Technology* — repositório em github.com/Applied-Language-Technology. Baixe-os de lá e salve na pasta `data/`.

Nota: os caminhos no material (ex.: `open('data/NYT_1991-01-16-A15.txt')`) supõem que você está rodando o Python **na pasta que contém `data/`**.

6. Rodar os exemplos

Cada unidade traz blocos de código para você experimentar. A forma mais confortável é o **Jupyter**:

```
jupyter lab
```

Isso abre o Jupyter no navegador; crie um notebook novo e cole os blocos de código da unidade, executando célula por célula (**Shift + Enter**). Você também pode salvar o código em um arquivo `.py` e rodar com `python3 arquivo.py`.

7. Rodar os exercícios (correção automática)

Cada unidade tem um conjunto de exercícios com **testes automáticos**, na pasta `exercicios/`. Edite o arquivo da unidade (ex.: `unidade1_exercicios.py`), substituindo cada `todo(...)` pela sua solução, e rode:

```
cd exercicios
python3 -m unittest unidade1_test
```

Quando aparecer **OK**, você acertou todos. Enquanto houver `todo(...)`, o teste falha de propósito.

Bom saber: os testes dos exercícios usam **só a biblioteca padrão** do Python — não precisam do spaCy nem do pandas. Assim, eles rodam em qualquer lugar, inclusive em corretores automáticos como o **Judge0**.

Checklist final

Antes de ir para a Unidade 1, confirme:

- ☐ `python3 --version` mostra **3.9 ou mais novo**
- ☐ o ambiente virtual está **ativado** (`.venv` aparece no terminal)
- ☐ `pip install -r requirements.txt` rodou sem erros
- ☐ `python -c "import spacy; spacy.load('en_core_web_sm')"` imprime **spaCy OK**
- ☐ a pasta `data/` contém os arquivos do corpus
- ☐ `python3 -m unittest unidade1_test` roda (mesmo que os exercícios ainda falhem)

Tudo certo? **Vá para a Unidade 1 — Manipulando texto com Python.**

Manipulando texto com Python

Trabalhando com Texto em Python I · Unidade 1

CiberExt 26-29 · FEELT38103 · Universidade Federal de Uberlândia

2026

Sumário

1. Computadores e texto	1
2. Codificação de texto (<i>text encoding</i>)	2
3. Carregando arquivos de texto simples no Python	3
4. Manipulando texto	5
5. Manipulando texto em escala	8
Quiz	12
Resumo da unidade	13

Objetivos de aprendizagem

Ao final desta unidade, você deverá ser capaz de:

- **compreender a diferença** entre texto formatado (*rich text*), texto estruturado (*structured text*) e texto simples (*plain text*);
- **entender o conceito** de codificação de caracteres (*text encoding*);
- **carregar arquivos** de texto simples no Python e **manipular** o seu conteúdo;
- **definir padrões flexíveis** de busca com expressões regulares;
- **processar vários arquivos** de uma só vez e salvar os resultados.

1. Computadores e texto

Os computadores podem armazenar e representar texto em formatos diferentes. Conhecer a distinção entre esses tipos é **fundamental** para processá-los programaticamente, porque cada formato traz consigo informações (ou ruídos) distintos.

1.1 O que é texto formatado (*rich text*)?

Processadores de texto, como o Microsoft Word, produzem **texto formatado** (*rich text*): texto cuja aparência foi estilizada de alguma maneira específica.

O texto formatado permite definir estilos visuais para os elementos do documento. Títulos, por exemplo, podem usar uma fonte diferente da do corpo do texto, que por sua vez pode empregar itálico ou negrito para dar ênfase. O texto formatado também pode incluir imagens, tabelas e outros elementos.

Esse é o formato padrão dos processadores de texto modernos do tipo *WYSIWYG* (*what you see is what you get* — “o que você vê é o que você obtém”).

1.2 O que é texto simples (*plain text*)?

Diferentemente do texto formatado, o **texto simples** (*plain text*) não contém nenhuma informação sobre a aparência visual: ele é feito **apenas de caracteres**.

Neste contexto, “caracteres” abrange letras, números, sinais de pontuação, espaços e quebras de linha. A definição de texto simples é um tanto flexível, mas, em geral, refere-se a texto sem qualquer informação de formatação ou estilo.

1.3 O que é texto estruturado (*structured text*)?

O **texto estruturado** (*structured text*) pode ser entendido como um caso especial de texto simples que inclui sequências de caracteres usadas para **formatar o texto para exibição**.

São exemplos de texto estruturado os textos descritos por linguagens de marcação como XML, Markdown ou HTML.

O exemplo abaixo mostra uma frase em texto simples envolvida por marcadores (*tags*) HTML de parágrafo `<p>`. A marca de abertura `<p>` e a de fechamento `</p>` informam ao computador que todo o conteúdo colocado entre elas forma um parágrafo:

```
<p>This is an example sentence.</p>
```

Essa informação é usada para estruturar o texto simples no momento de renderizá-lo para exibição, normalmente aplicando estilo à sua aparência.

1.4 Por que isso importa?

Ao reunir um conjunto de textos para formar um **corpus**, é bem provável que parte deles tenha origem em formato formatado ou estruturado, dependendo do meio de onde vieram:

- Se você coleta documentos impressos que foram digitalizados por **reconhecimento óptico de caracteres** (*optical character recognition*, OCR) e depois convertidos de texto formatado para texto simples, a remoção das informações de formatação tende a **introduzir erros** no texto resultante. Trabalhar com esse tipo de OCR “sujo” pode afetar os resultados da análise textual (Hill & Hengchen, 2019).
- Se você coleta documentos digitais raspando fóruns de discussão ou sites, é provável que encontre vestígios de texto estruturado na forma de marcadores de marcação, que podem ser arrastados para o texto simples durante a conversão.

O texto simples é, de longe, o formato **mais interoperável** para texto, por ser fácil de ler para os computadores. É por isso que as linguagens de programação trabalham com texto simples — e, se você pretende usar programação para manipular texto, precisa saber o que é texto simples.

Em resumo: ao trabalhar com texto simples, muitas vezes será preciso lidar com os vestígios deixados pela conversão a partir de texto formatado ou estruturado.

2. Codificação de texto (*text encoding*)

Para serem lidos pelos computadores, os textos simples precisam ser **codificados**. Isso é feito por meio da **codificação de caracteres** (*character encoding*), que mapeia cada caractere (letras, números, pontuação, espaços. . .) para uma representação numérica compreendida pela máquina.

O ideal seria não termos de lidar com operações de baixo nível como a codificação de caracteres — mas, na prática, precisamos, porque existem **vários sistemas** de codificação e eles **não são compatíveis entre si**. Essa é a origem de boa parte das dores de cabeça de quem trabalha com texto simples.

Há dois sistemas de codificação com os quais você provavelmente vai se deparar: **ASCII** e **Unicode**.

2.1 ASCII

O **ASCII** (*American Standard Code for Information Interchange*) é um sistema pioneiro de codificação de caracteres que serviu de base para muitos sistemas modernos.

O ASCII ainda é bastante usado, mas é **muito limitado** quanto à variedade de caracteres. Se o seu idioma inclui caracteres como **ä, ö** — ou os nossos **á, ç, ã** —, o ASCII não dá conta.

2.2 Unicode

O **Unicode** é um padrão para codificar texto na maioria dos sistemas de escrita usados no mundo, cobrindo cerca de **140 mil caracteres** entre escritas modernas e históricas, símbolos e emojis.

Por exemplo, o emoji de fatia de pizza tem o “código” Unicode **U+1F355**, enquanto o código correspondente a um espaço em branco é **U+0020**.

O Unicode pode ser implementado por diferentes codificações, como o **UTF-8**, definido pelo próprio padrão Unicode.

O UTF-8 é **retrocompatível com o ASCII**. Em outras palavras, as codificações do ASCII formam um subconjunto do UTF-8 — o que facilita bastante a nossa vida. Assim, mesmo que um arquivo de texto simples tenha sido codificado em ASCII, podemos decodificá-lo como UTF-8; **o contrário, porém, não vale**.

3. Carregando arquivos de texto simples no Python

Arquivos de texto simples podem ser carregados no Python com a função `open()`.

O primeiro argumento de `open()` deve ser uma *string* contendo o **caminho** para o arquivo que será aberto. Aqui, temos um caminho que aponta para um arquivo chamado `NYT_1991-01-16-A15.txt`, localizado em um diretório chamado `data`. Na definição do caminho, o diretório e o nome do arquivo são separados por uma barra `/`.

Para acessar o arquivo apontado por esse caminho, passamos o caminho como *string* ao argumento `file` da função `open()`:

```
open(file='data/NYT_1991-01-16-A15.txt', mode='r', encoding='utf-8')
```

Vale entender os demais argumentos:

- Por padrão, o Python 3 assume que o texto está codificado em **UTF-8**, mas podemos deixar isso explícito com o argumento `encoding`, passando a *string* `utf-8`.

- O argumento `mode` define o que queremos fazer com o arquivo. A *string* `r` (de *read*) indica que queremos **apenas ler** o arquivo.
- Usamos `open()` junto da instrução `with`, que garante que o arquivo será **fechado** após executarmos o que precisamos dentro do bloco de código indentado que segue o `with`. Isso evita que o arquivo continue consumindo memória e recursos depois que não precisamos mais dele.

```
# Abre um arquivo e o atribui à variável 'file'
with open(file='data/NYT_1991-01-16-A15.txt', mode='r', encoding='utf-8') as file:

    # A instrução 'with' deve ser seguida por um bloco de código indentado.
    # Aqui chamamos o método read() para ler o conteúdo do arquivo e
    # atribuímos o resultado à variável 'text'.
    text = file.read()
```

Repare que `with` e `open()` são seguidos da palavra `as` e de uma variável chamada `file`. Isso instrui o Python a atribuir à variável `file` aquilo que `open()` retornar.

Se agora chamarmos a variável `file`, obtemos um objeto Python do tipo `TextIOWrapper`, que carrega três informações: o caminho do arquivo (no argumento `name`) e os argumentos `mode` e `encoding` que definimos acima:

```
# Chama a variável para examinar o objeto
file
```

```
<_io.TextIOWrapper name='data/NYT_1991-01-16-A15.txt' mode='r' encoding='utf-8'>
```

Tenha em mente que, no bloco indentado após o `with`, chamamos o método `read()` do objeto `TextIOWrapper`. Esse método leu o conteúdo do arquivo, que atribuímos à variável `text`.

No entanto, se tentarmos chamar `read()` para a variável `file` **fora** do bloco `with`, o Python levanta um erro, porque o arquivo já foi fechado:

```
# Tenta usar o método read() para ler o conteúdo do arquivo
file.read()
```

```
ValueError: I/O operation on closed file.
```

Esse comportamento é o esperado: queremos que o arquivo seja fechado, para que não consuma memória nem recursos quando não é mais necessário. Isso é especialmente importante ao trabalhar com **milhares de arquivos**, pois cada arquivo aberto ocupa memória.

Vejamos agora o resultado de `read()`, guardado em `text`. O texto é bem longo, então vamos pegar apenas uma **fatia** com os primeiros 500 caracteres, usando colchetes `[ : 500 ]`. Adicionar colchetes logo após o nome de uma variável permite acessar partes do objeto, quando o objeto permite isso. Por exemplo, `text[1]` recuperaria o caractere na posição 1; o `:` antes do número instrui o Python a recuperar **todos** os caracteres até o de posição 500.

```
# Recupera os primeiros 500 caracteres da variável 'text'
text[:500]
```

```
'U.S. TAKING STEPS TO CURB TERRORISM: F.B.I. Is Ordered to Find Iraqis Whose Visas Have
↳ Expired\nBy JAMES BARRON\nNew York Times (1923-Current file); Jan 16, 1991; ... the
↳ Justice Department'
```

A maior parte do texto está legível, mas há sequências estranhas, como `bem` no início e vários `\n` ao longo do texto:

- A sequência (código `U+FEFF`) é o **BOM** (*byte order mark*, “marca de ordem de bytes”). Ele foi criado para indicar a ordem dos bytes em codificações como UTF-16; **em UTF-8 não há ordem de bytes**, então o BOM é **desnecessário** e seu uso é **desaconselhado** pelo próprio padrão Unicode. Ainda assim, alguns programas e editores (especialmente no Windows) o adicionam no início do arquivo como uma espécie de “assinatura” de UTF-8. É por isso que ele aparece aqui como um caractere fantasma no começo do texto — e, se não for removido, pode causar bugs (por exemplo, a primeira palavra deixa de casar em comparações). Nem todo arquivo UTF-8 contém essa marca.
- As sequências `\n` indicam **quebra de linha**.

Isso fica evidente se usarmos a função `print()` para imprimir os primeiros 1000 caracteres:

```
# Imprime os primeiros 1000 caracteres da variável 'text'
print(text[:1000])
```

```
U.S. TAKING STEPS TO CURB TERRORISM: F.B.I. Is Ordered to Find Iraqis Whose Visas Have
→ Expired
By JAMES BARRON
New York Times (1923-Current file); Jan 16, 1991;
ProQuest Historical Newspapers: The New York Times with Index pg. A15
...
    The Federal Bureau of Investigation has been ordered to track down as many as 3,000
→ Iraqis in this country whose visas have expired, the Justice Department said
→ yesterday.
```

Como se vê, o Python sabe interpretar as sequências `\n` e insere uma quebra de linha ao encontrá-las durante a impressão. Note também que as primeiras linhas do arquivo contêm **metadados** do artigo (nome, autor e fonte), que antecedem o corpo do texto.

4. Manipulando texto

Como todo o conteúdo guardado em `text` é um objeto *string*, podemos usar todos os métodos de manipulação de *strings* do Python.

4.1 Substituindo com `replace()`

Vamos usar o método `replace()` para trocar todas as quebras de linha `"\n"` por *strings* vazias `""` e guardar o resultado em `processed_text`:

```
# Substitui as quebras de linha \n por strings vazias e atribui o resultado
# à variável 'processed_text'
processed_text = text.replace('\n', '')

# Imprime os primeiros 1000 caracteres da variável 'processed_text'
print(processed_text[:1000])
```

Agora todo o texto ficou “grudado”. Ainda assim, conseguimos identificar o início de cada parágrafo, marcado por **três espaços em branco**.

Atenção: remover as quebras de linha também fez os metadados do artigo se fundirem em

um único parágrafo. Sempre fique atento aos **efeitos indesejados** de substituições e outras transformações!

4.2 Dividindo com `split()`

Se nos interessa apenas o corpo do artigo, podemos remover os metadados com facilidade, pois sabemos que eles estão separados do corpo por três espaços. A maneira mais simples é usar `split()` para **dividir a string em uma lista**, usando os três espaços como separador:

```
# Usa o método split() com três espaços como separador. Atribui o
# resultado à variável 'processed_text'.
processed_text = processed_text.split(sep='  ')

# Imprime o resultado da variável 'processed_text'
print(processed_text)
```

O método `split()` retorna uma **lista** de objetos *string*. Podemos confirmar isso verificando o tipo do objeto:

```
# Verifica o tipo do objeto na variável 'processed_text'
type(processed_text)
```

```
list
```

Os metadados ficaram no **primeiro item** da lista, pois a primeira sequência de três espaços aparece justamente onde os metadados terminam. Vamos buscar esse primeiro item — lembre que o Python começa a contar do zero, então ele está no índice **0**:

```
# Recupera o objeto string no índice 0 da lista 'processed_text'
processed_text[0]
```

4.3 Removendo itens com `pop()`

Para remover os metadados e manter apenas o corpo do texto, usamos o método `pop()` da lista. Ele espera um número inteiro: o índice do item a ser removido.

```
# Chama o método pop() da lista 'processed_text' com o
# índice do item a ser removido.
processed_text.pop(0)
```

Você pode se perguntar por que **não** atribuímos o resultado a uma variável. A resposta é que as listas do Python são **mutáveis**, ou seja, podem ser alteradas “no lugar” (*in place*). O método `pop()` modifica a lista sem precisar reatribuir o valor à variável. Podemos conferir recuperando os três primeiros itens:

```
# Recupera os três primeiros itens da lista 'processed_text'
processed_text[:3]
```

O primeiro item da lista **não corresponde mais** aos metadados.

4.4 Reunindo com `join()`

Para converter a lista de volta em uma *string*, usamos o método `join()` de um objeto *string*. O `join()` espera um **iterável** como entrada (algo que possa ser percorrido, como uma lista ou um dicionário).

Aqui pode haver certa confusão: o `join()` deve ser chamado **sobre a *string* que servirá de “cola”** entre os itens do iterável. No nosso caso, queremos usar como cola a sequência original que separava os parágrafos — uma quebra de linha e três espaços (`'\n '`):

```
# Usa o método join() para unir os itens da lista 'processed_text'
# usando a string '\n   ' - uma quebra de linha e três espaços. Guarda o
# resultado na variável de mesmo nome.
processed_text = '\n   '.join(processed_text)

# Verifica o resultado imprimindo os primeiros 1000 caracteres da string
# resultante em 'processed_text'
print(processed_text[:1000])
```

Aplicar o `join()` devolve uma *string* com as quebras de parágrafo originais!

4.5 Um “pipeline” de substituições com `laço for`

Examinando o texto de perto, dá para ver resquícios da digitalização: o OCR resultou em uma **mistura de aspas** de tipos diferentes, como `"`, `'`, `”`, `‘` e `”`. Se quiséssemos extrair citações do corpo do texto, seria bom padronizar as aspas. Vamos escolher `"` (uma aspa dupla simples) como padrão.

Poderíamos aplicar `replace()` separadamente para cada tipo de aspa, mas isso seria tedioso. Para tornar o processo mais eficiente, combinamos duas estruturas de dados do Python: **listas e tuplas**.

Começamos definindo uma lista chamada `pipeline`. Criamos e preenchemos uma lista simplesmente colocando objetos entre colchetes `[]`, separados por vírgula. Como `replace()` recebe duas *strings*, combinamos cada par em uma **tupla** — estruturas finitas e ordenadas, marcadas por parênteses `()`. Em cada tupla, colocamos o caractere a ser substituído na primeira *string* e o substituto na segunda:

```
# Define uma lista com quatro tuplas, cada uma com duas strings: o caractere
# a ser substituído e o seu substituto.
pipeline = [('"', '"'), ('‘', '’'), ('”', '”'), ('’', '’')]
```

Isso ilustra como diferentes estruturas de dados costumam ser **aninhadas** no Python: a lista contém tuplas, e as tuplas contém *strings*.

Agora podemos percorrer cada item da lista com um `laço for`, que itera pelos itens na ordem em que aparecem. Cada item é uma tupla com duas *strings*. Para entrar no `laço`, o Python espera que a próxima linha esteja **indentada** (use a tecla `Tab` →):

```
# Percorre as tuplas da lista 'pipeline'. Cada tupla tem dois valores, que
# atribuímos as variáveis 'old' e 'new' automaticamente!
for old, new in pipeline:

    # Usa o método replace() para substituir a string da variável 'old'
    # pela string da variável 'new'
    processed_text = processed_text.replace(old, new)
```

O que acontece dentro do `laço` é exatamente o que fizemos antes com `replace()`, mas, em vez de definir manualmente as *strings*, usamos as *strings* contidas nas variáveis `old` e `new`! A cada volta, atualizamos automaticamente a *string* em `processed_text`.

```
# Imprime a string
print(processed_text)
```

Conseguimos realizar uma série de substituições percorrendo a lista de tuplas, que definia os padrões a substituir e seus substitutos.

Recapitulando a sintaxe do `for`: declaramos o início do laço com `for`, seguido de uma variável usada para se referir aos itens obtidos da lista. A lista percorrida vem precedida de `in` e do nome da variável atribuída à lista inteira.

Para entender melhor, vejamos o que acontece se definirmos **apenas uma** variável, `our_tuple`, para os itens obtidos:

```
# Percorre os itens da variável 'pipeline'
for our_tuple in pipeline:

    # Imprime o objeto retornado
    print(our_tuple)
```

```
('...', '...')
('...', '...')
('...', '...')
('...', '...')
```

Isso imprime as **tuplas** inteiras! O Python é esperto o bastante para entender que uma única variável se refere aos itens (as tuplas) da lista, ao passo que, com duas variáveis, ele avança até as *strings* contidas dentro de cada tupla. Ao escrever laços `for`, preste muita atenção aos itens contidos na lista!

5. Manipulando texto em escala

O ideal é que o Python permita manipular texto **em escala**, isto é, aplicar o mesmo procedimento a dez, cem ou mil arquivos com o mesmo esforço. Para isso, precisamos definir padrões **mais flexíveis** do que as *strings* fixas usadas com `replace()`, além de abrir e fechar arquivos automaticamente. Essas capacidades vêm dos módulos de **expressões regulares** e de **manipulação de arquivos**.

5.1 Expressões regulares (*regular expressions*)

As expressões regulares são uma “linguagem” que permite definir **padrões de busca**. Esses padrões podem ser usados para encontrar (ou encontrar e substituir) trechos em objetos *string*. Diferentemente das *strings* fixas, elas permitem definir **curingas** (caracteres que representam qualquer caractere), **quantificadores** (que casam sequências repetidas) e muito mais.

O Python oferece expressões regulares por meio do módulo `re`, que ativamos com o comando `import`:

```
import re
```

Vamos carregar um arquivo, ler seu conteúdo, atribuir os **últimos** 2000 caracteres à variável `extract` e imprimir o resultado:

```
# Define o caminho do arquivo e o abre para leitura (r) com codificação utf-8
with open(file='data/WP_1990-08-10-25A.txt', mode='r', encoding='utf-8') as file:

    # Lê o conteúdo do arquivo com o método .read()
    text = file.read()
```

```
# Pega os *últimos* 2000 caracteres - note o sinal de menos antes do número
extract = text[-2000:]

# Imprime o resultado
print(extract)
```

O texto tem muitos erros de OCR, principalmente sequências como `....` e `"""`.

Vamos **compilar** nossa primeira expressão regular, que busca sequências de dois ou mais pontos finais, com a função `compile()` do módulo `re`. Ela recebe uma *string* como entrada. Note o prefixo `r` antes da *string*: ele diz ao Python para guardar a *string* em formato “bruto” (*raw*), ou seja, exatamente como aparece.

```
# Compila uma expressão regular e a atribui a variável 'stops'
stops = re.compile(r'\.{2,}')
```

```
# Vamos verificar o tipo da expressão regular!
type(stops)
```

```
re.Pattern
```

Vamos destrinchar essa expressão:

- Ela é definida por uma *string* do Python (aspas simples `' '`).
- Precisamos de uma **barra invertida** `\` antes do ponto final `.`, porque, nas expressões regulares, o ponto final é um **curinga** que representa qualquer caractere. A barra diz ao Python que queremos o ponto final “de verdade”.
- As **chaves** `{ }` instruem a expressão a buscar ocorrências do item anterior (`\.`, nosso ponto de verdade) que aconteçam **duas ou mais vezes** (`2,`). Isso preserva os usos legítimos de um único ponto final.

Em português claro: buscamos ocorrências de **dois ou mais pontos finais**.

Para aplicar a expressão a algum texto, usamos o método `sub()` do objeto recém-criado `stops`. O `sub()` recebe dois argumentos:

- `repl`: a *string* usada para **substituir** as ocorrências encontradas;
- `string`: o objeto *string* onde procurar as ocorrências.

O método retorna a *string* modificada:

```
# Aplica a expressão regular ao texto em 'extract' e salva a saída
# na mesma variável, essencialmente sobrescrevendo o texto antigo.
extract = stops.sub(repl='', string=extract)
```

```
# Imprime o texto para examinar o resultado
print(extract)
```

As sequências de pontos finais desapareceram.

5.2 Alternativas na expressão regular

Podemos tornar a expressão mais poderosa acrescentando **alternativas**. Vamos compilar outra e guardá-la em `punct`:

```
# Compila uma expressão regular e a atribui a variável 'punct'
punct = re.compile(r'(\.|,){2,}')
```

A novidade são os **parênteses** () e a **barra vertical** | entre eles, que separa o ponto final \. da vírgula ,. Caracteres entre parênteses e separados por | marcam **alternativas**. Em português claro: buscamos ocorrências de **dois ou mais pontos finais ou vírgulas**.

Para garantir que o padrão funcione como esperado, recuperamos o texto original de `text` e o reatribuímos a `extract`, sobrescrevendo as edições anteriores:

```
# "Reinicia" a variável extract pegando os últimos 2000 caracteres da string original
extract = text[-2000:]

# Aplica a expressão regular
extract = punct.sub(repl='', string=extract)

# Imprime o resultado
print(extract)
```

Sucesso! As sequências de pontos finais e de vírgulas podem ser removidas com uma única expressão regular.

Sequências mais irregulares vindas de erros de OCR — como `'-*', ->.', /*-.` — são bem mais difíceis de capturar e exigiriam expressões mais complexas. Essa complexidade é justamente o que torna as expressões regulares tão poderosas, mas aprender a usá-las leva **tempo e paciência**.

Dica: use um serviço como regex101.com para aprender e testar expressões regulares interativamente.

Na prática, criar expressões que cubram o máximo de casos é particularmente difícil. Capturar a maioria dos erros — talvez distribuindo as transformações em uma **sequência de etapas** (um *pipeline*) — já ajuda muito a preparar o texto para análise. Lembre-se, porém: para identificar padrões de manipulação de forma confiável, você deve sempre observar **mais de um** texto do seu corpus.

5.3 Processando múltiplos arquivos

Muitos corpora contêm textos em vários arquivos. Para manipular grandes volumes de texto com eficiência, precisamos abrir os arquivos, ler seu conteúdo, executar as operações desejadas e fechá-los — tudo **programaticamente**.

Esse procedimento fica bem simples com a classe `Path` do módulo `pathlib` do Python. Usar `from ... import ...` permite importar **apenas parte** de um módulo — aqui, só a classe `Path`:

```
from pathlib import Path
```

A classe `Path` codifica informações sobre caminhos em uma estrutura de diretórios. O grande trunfo dela é **inferir automaticamente** o tipo de caminho usado pelo seu sistema operacional. Isso importa porque Windows, Linux e macOS usam caminhos de arquivo diferentes; a classe `Path` nos poupa de muita dor de cabeça, sobretudo se quisermos que o código rode em sistemas diferentes.

Nosso repositório contém um diretório chamado `data`, com os arquivos de texto que viemos usando. Vamos inicializar um objeto `Path` apontando para esse diretório e atribuí-lo à variável `corpus_dir`:

```
# Cria um objeto Path que aponta para o diretório 'data' e o atribui
# a variável 'corpus_dir'
corpus_dir = Path('data')
```

O objeto `Path` tem vários métodos e atributos úteis. Podemos, por exemplo, verificar se o caminho é válido com `exists()`, que retorna um valor **booleano** (`True` ou `False`):

```
# Usa o método exists() para verificar se o caminho é válido
corpus_dir.exists()
```

```
True
```

Também podemos checar se o caminho é um diretório com `is_dir()`, e garantir que ele **não** aponta para um arquivo com `is_file()`:

```
# Usa o método is_dir() para verificar se o caminho aponta para um diretório
corpus_dir.is_dir()
```

```
True
```

```
# Usa o método is_file() para verificar se o caminho aponta para um arquivo
corpus_dir.is_file()
```

```
False
```

Sabendo que o caminho aponta para um diretório, usamos o método `glob()` para coletar todos os arquivos de texto. O nome `glob` vem de *global* e foi implementado originalmente como um programa para casar nomes de arquivos e caminhos usando curingas. O `glob()` exige o argumento `pattern`, que recebe uma *string*: o `*` (asterisco) atua como **curinga**, representando qualquer sequência de caracteres antes de `.txt` (sufixo comum de arquivos de texto simples). Convertendo o resultado em lista com `list()`, podemos percorrê-lo facilmente:

```
# Coleta todos os arquivos com sufixo .txt no diretório 'corpus_dir' e converte o
# resultado em uma lista
files = list(corpus_dir.glob(pattern='*.txt'))

# Mostra o resultado
files
```

```
[PosixPath('data/WP_1990-08-10-25A.txt'),
 PosixPath('data/NYT_1991-01-16-A15.txt'),
 PosixPath('data/WP_1991-01-17-A1B.txt')]
```

Temos uma lista de três objetos `Path` apontando para três arquivos! Isso nos permite percorrê-los com um laço `for` e manipular o texto de cada um. No bloco abaixo, iteramos sobre cada arquivo, lemos e modificamos seu conteúdo e o gravamos em um novo arquivo:

```
# Percorre a lista de objetos Path em 'files'. Refere-se a cada arquivo
# pela variável 'file'.
for file in files:

    # Usa o método read_text() de um objeto Path para ler o conteúdo do arquivo.
    # Passa o valor 'utf-8' ao argumento 'encoding' para declarar a codificação.
```

```

# Guarda o resultado na variável 'text'.
text = file.read_text(encoding='utf-8')

# Aplica a expressão regular definida acima para remover a pontuação
# excessiva do texto. Guarda o resultado na variável 'mod_text'.
mod_text = punct.sub(' ', text)

# Define um novo nome de arquivo com o prefixo 'mod_' criando uma nova string.
# O objeto Path guarda o nome do arquivo como string no atributo 'name'.
# Combina as duas strings com o operador '+'.
new_filename = 'mod_' + file.name

# Define um novo objeto Path que aponta para o novo arquivo. O objeto Path
# junta automaticamente o diretório e o nome do arquivo para nós.
new_path = Path('data', new_filename)

# Imprime uma mensagem de status usando formatação de string. Ao adicionar
# o prefixo 'f' a uma string, podemos usar chaves {} para inserir uma
# variável dentro da string. Aqui inserimos o caminho atual do arquivo.
print(f'Writing modified text to {new_path}')

# Usa o método write_text() para escrever o texto modificado de 'mod_text'
# no arquivo, com codificação UTF-8.
new_path.write_text(mod_text, encoding='utf-8')

```

```

Writing modified text to data/mod_WP_1990-08-10-25A.txt
Writing modified text to data/mod_NYT_1991-01-16-A15.txt
Writing modified text to data/mod_WP_1991-01-17-A1B.txt

```

Como se vê, os objetos `Path` oferecem dois métodos convenientes para trabalhar com arquivos de texto: `read_text()` e `write_text()`. Eles permitem ler e gravar texto **sem** a instrução `with` — e, assim como o `with`, o arquivo apontado pelo `Path` é fechado automaticamente após a leitura.

Quiz

Marque a alternativa correta (a resposta certa está destacada com ✓).

1. Qual formato de texto é o mais interoperável para programação?

1. Texto formatado (*rich text*)
2. Texto estruturado (*structured text*)
3. Texto simples (*plain text*) ✓

2. Num texto, a sequência de dois caracteres “barra-n” representa:

1. Um espaço em branco
2. Uma quebra de linha ✓
3. Uma tabulação

3. O UTF-8 é retrocompatível com qual codificação?

1. ASCII ✓
2. Latin-1

3. UTF-16

4. Qual método de string divide o texto em uma lista?

1. `join()`
2. `split()` ✓
3. `replace()`

5. Numa expressão regular, o quantificador “dois ou mais” é escrito como:

1. `{2}`
2. `{2,}` ✓
3. `{,2}`

6. Qual método da classe `Path` coleta arquivos por um padrão com curinga (ex.: todos os `.txt`)?

1. `glob()` ✓
2. `read_text()`
3. `exists()`

Resumo da unidade

Nesta unidade, você aprendeu a:

1. **distinguir** texto formatado, estruturado e simples, e a entender por que o texto simples é o formato preferido em programação;
2. **compreender** a codificação de caracteres (ASCII e Unicode/UTF-8) e por que ela causa problemas;
3. **carregar** arquivos com `open()` + `with` + `read()`, lidando com marcas como `e` e `\n`;
4. **manipular strings** com `replace()`, `split()`, `pop()` e `join()`, e montar um *pipeline* de substituições com listas, tuplas e laços `for`;
5. **definir padrões flexíveis** com expressões regulares (`re.compile()`, `sub()`, curingas, quantificadores e alternativas);
6. **processar vários arquivos** de uma vez com `pathlib.Path`, `glob()`, `read_text()` e `write_text()`.

Exercícios sugeridos

1. Carregue um dos arquivos do diretório `data` e conte quantas quebras de linha (`\n`) ele contém antes de qualquer limpeza.
2. Escreva um *pipeline* (lista de tuplas) que padronize **travessões** (`-`, `-`, `-`) em um único caractere.
3. Crie uma expressão regular que remova sequências de dois ou mais espaços em branco, substituindo-as por um único espaço.
4. Adapte o laço da Seção 5.3 para gravar os arquivos modificados em um novo diretório chamado `output` em vez de `data`.

Referências

- Rögnavaldsson, E. et al. *Applied Language Technology* (MOOC). Universidade de Helsinque. <https://applied-language-technology.mooc.fi/>
- Hill, M. J.; Hengchen, S. (2019). Quantifying the impact of dirty OCR on historical text analysis. *Digital Scholarship in the Humanities*.

Processando textos com spaCy

Trabalhando com Texto em Python I · Unidade 2

CiberExt 26-29 · FEELT38103 · Universidade Federal de Uberlândia

2026

Sumário

1. Primeiros passos	1
2. Realizando tarefas básicas de PLN com spaCy	2
Quiz	8
Resumo da unidade	9

Objetivos de aprendizagem

Ao final desta unidade, você deverá:

- conhecer alguns dos **conceitos e tarefas fundamentais** do processamento de linguagem natural (PLN);
- saber **realizar tarefas simples** de PLN usando a biblioteca **spaCy**.

1. Primeiros passos

Para começar, importamos o **spaCy**, uma das muitas bibliotecas disponíveis para processamento de linguagem natural em Python.

```
# Importa a biblioteca spaCy
import spacy
```

Dica prática: a primeira importação do spaCy pode demorar alguns segundos, porque a biblioteca carrega dependências pesadas (redes neurais, tokenizadores etc.). Isso é normal.

Para realizar tarefas de processamento de linguagem natural em um idioma específico, precisamos carregar um **modelo de linguagem** treinado para executar essas tarefas naquele idioma.

O spaCy oferece suporte a muitos idiomas, mas disponibiliza **modelos pré-treinados** apenas para um subconjunto deles. Esses modelos vêm em tamanhos e “sabores” diferentes — exploraremos essas diferenças mais adiante no curso. Para nos familiarizarmos com as tarefas básicas de PLN, começaremos com um modelo pequeno para o **inglês**.

Os modelos de linguagem são carregados com a função `load()` do spaCy, que recebe o **nome do modelo** como entrada:

```
# Carrega o modelo pequeno de língua inglesa e o atribui a variável 'nlp'
nlp = spacy.load('en_core_web_sm')

# Chama a variável para examinar o objeto
nlp
```

Chamar a variável `nlp` retorna um objeto `Language` do spaCy, que contém o modelo de linguagem para o inglês. Em essência, o objeto `Language` do spaCy é um **pipeline** (uma sequência de etapas de processamento) que usa o modelo de linguagem para executar diversas tarefas de PLN — tarefas que veremos a seguir.

1.1 O que é um modelo de linguagem?

A maioria dos modelos de linguagem modernos é baseada em **estatística**, e não em regras definidas manualmente por humanos. Modelos de linguagem estatísticos se apoiam em **probabilidades**, respondendo perguntas como:

- Qual a probabilidade de uma determinada frase ocorrer em um idioma?
- Qual a probabilidade de uma determinada palavra ocorrer em uma sequência de palavras?

Considere as frases a seguir, extraídas dos artigos de jornal usados na unidade anterior:

*“From financial exchanges in **OCULTO** Manhattan to cloakrooms in Washington and homeless shelters in California, unfamiliar rituals were the order of the day.”*

*“Security precautions were being taken around the **OCULTO** as the deadline for Iraq to withdraw from Kuwait neared.”*

Você provavelmente consegue **arriscar um bom palpite** sobre as palavras ocultas, com base no seu conhecimento da língua inglesa e do mundo em geral (no primeiro caso, algo como *“lower”*; no segundo, *“country”*).

Da mesma forma, construir um modelo de linguagem estatístico envolve observar a ocorrência de palavras em **grandes corpora** e calcular suas probabilidades de ocorrência em um determinado contexto. O modelo é então treinado fazendo previsões e ajustando-se com base nos erros cometidos durante essas previsões.

1.2 Como os modelos de linguagem são treinados?

O modelo pequeno para o inglês, por exemplo, é treinado em um corpus chamado **OntoNotes 5.0**, que reúne textos de gêneros variados — notícias de agências de imprensa, notícias de telejornal, conversas telefônicas e transmitidas, e blogs. Isso permite que o corpus cubra a variação linguística tanto da língua **escrita** quanto **falada** em inglês.

O corpus OntoNotes 5.0 é composto por mais do que texto puro: suas anotações incluem **classes gramaticais** (*part-of-speech tags*), **dependências sintáticas** e **correferências** entre palavras. Isso permite modelar não apenas a ocorrência de palavras ou sequências de palavras, mas também suas **características gramaticais**.

2. Realizando tarefas básicas de PLN com spaCy

Para processar um texto usando o objeto `Language` que contém o modelo de linguagem para o inglês, basta **chamar** o objeto `nlp` sobre algum texto.

Vamos definir uma frase de teste simples — um objeto *string* do Python, guardado na variável `text`:

```
# Atribui uma frase de exemplo a variável 'text'
text = "The Federal Bureau of Investigation has been ordered to track down as many as
→ 3,000 Iraqis in this country whose visas have expired, the Justice Department said
→ yesterday."

# Chama a variável para examinar o resultado
text
```

Passar a variável `text` para o objeto `Language nlp` retorna um objeto `Doc` do spaCy (abreviação de *document*, “documento”). Em processamento de linguagem natural, trechos maiores de texto costumam ser chamados de “documentos”, ainda que, neste caso, nosso documento consista em uma única frase.

Esse objeto contém tanto o texto de entrada (`text`) quanto os **resultados** do processamento de linguagem natural feito pelo spaCy.

```
# Alimenta a string em 'text' ao objeto Language em 'nlp'
# Guarda o resultado na variável 'doc'
doc = nlp(text)
```

O objeto `Doc` agora está guardado na variável `doc`:

```
# Chama a variável para examinar o objeto
doc
```

Embora a saída se pareça com uma *string* comum do Python, o objeto `Doc` contém uma **riqueza de informações** sobre a estrutura linguística do texto, geradas pelo spaCy ao processar a frase através do seu *pipeline* de PLN. Vamos agora examinar, uma a uma, as tarefas realizadas “nos bastidores”.

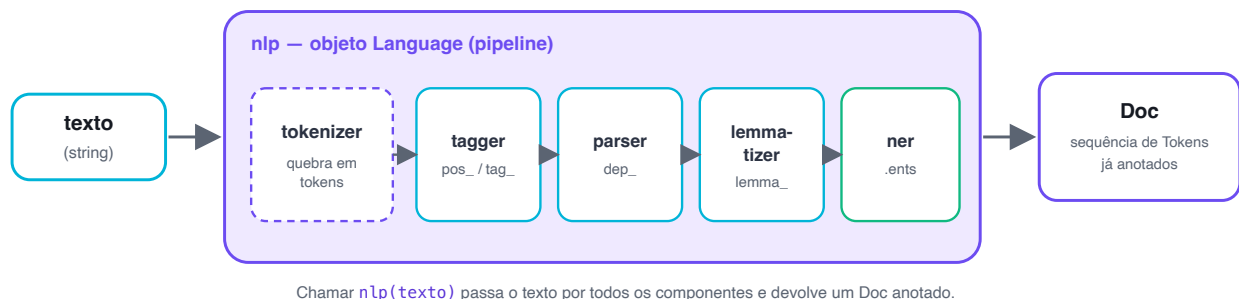


Figura 1: O *pipeline* de PLN do spaCy: chamar `nlp(texto)` passa o texto por vários componentes (tokenizer, tagger, parser, lematizador, NER) e devolve um objeto `Doc` já anotado.

2.1 Tokenização

A primeira tarefa realizada é conhecida como **tokenização**: ela quebra o texto em unidades analíticas que serão processadas posteriormente.

Na maioria dos casos, um **token** corresponde a uma palavra separada por espaço em branco, mas sinais de pontuação também são considerados tokens independentes. Como os computadores tratam palavras como seqüências de caracteres, atribuir aos sinais de pontuação seus próprios tokens evita que a pontuação final “grude” nas palavras que a precedem.

Um objeto `Doc` do spaCy é composto por uma sequência de objetos `Token`, que armazenam os resultados das diversas tarefas de PLN. Vamos imprimir cada objeto `Token` guardado em `doc`:

```
# Percorre os itens no objeto Doc, usando a variável 'token' para se referir aos itens da
→ lista
for token in doc:

    # Imprime cada token
    print(token)
```

A saída mostra um `Token` por linha. Como esperado, sinais de pontuação como `.` e `,` constituem seus próprios `Tokens`.

2.2 Anotação morfossintática (*part-of-speech tagging*)

A **anotação morfossintática** (*POS tagging*) é a tarefa de determinar a **classe gramatical** de um token. Isso é crucial para desambiguação, já que classes gramaticais diferentes podem ter formas idênticas.

Considere o exemplo em inglês: “*The sailor **dogs** the hatch.*” (algo como “O marinheiro tranca a escotilha”). O presente do verbo *to dog* (prender/trancar algo com algo) é exatamente igual ao plural do substantivo *dog* (cachorro): **dogs**. Para identificar a classe correta, é preciso examinar o **contexto** em que a palavra aparece.

O spaCy fornece dois tipos de anotação morfossintática, uma **genérica** (*coarse*) e uma **detalhada** (*fine-grained*), guardadas respectivamente nos atributos `pos_` e `tag_`. Acessamos os atributos de um objeto Python inserindo o atributo após o objeto, separados por um ponto final — por exemplo, `token.pos_`.

```
# Percorre os itens no objeto Doc, usando a variável 'token' para se referir aos itens da
→ lista
for token in doc:

    # Imprime o token e as anotações morfossintáticas genéricas e detalhadas
    print(token, token.pos_, token.tag_)
```

As anotações genéricas disponíveis em `pos_` são baseadas no conjunto de rótulos do projeto **Universal Dependencies**. Já as anotações detalhadas em `tag_` são baseadas no corpus OntoNotes 5.0 apresentado acima. Diferentemente das anotações genéricas, as detalhadas também codificam informação gramatical adicional — as anotações de verbos, por exemplo, se distinguem por **aspecto** e **tempo verbal**.

2.3 Análise morfológica

Morfemas são as menores unidades gramaticais que carregam significado. Reconhecem-se geralmente dois tipos: **morfemas livres**, que consistem em palavras capazes de existir sozinhas, e **morfemas presos**, que flexionam outros morfemas. No inglês, morfemas presos incluem sufixos como `-s`, usado para indicar o plural de um substantivo.

Em outras palavras, os morfemas moldam a forma externa de uma palavra, e essas formas estão associadas a funções gramaticais específicas.

O spaCy realiza a análise morfológica automaticamente, guardando o resultado no atributo `morph` de um objeto `Token`:

```
# Percorre os itens no objeto Doc, usando a variável 'token' para se referir aos itens da
→ lista
for token in doc:
```

```
# Imprime o token e o resultado da análise morfológica
print(token, token.morph)
```

Como mostra a saída, **nem todos os tokens** têm informação morfológica — alguns consistem em morfemas livres, sem flexão. Para recuperar informação morfológica de um **Token**, usamos o método `get()` do atributo `morph`. Podemos usar colchetes `[]` para acessar itens no objeto **Doc**. A linha a seguir recupera a informação morfológica de **aspecto** para o 23º token do **Doc** (índice 22):

```
# Recupera informação morfológica sobre aspecto para o Token no índice 22 do objeto Doc
doc[22].morph.get('Aspect')
```

Isso retorna uma lista com um único item, a *string* `Perf`, que se refere ao **aspecto perfectivo**.

O que acontece se tentarmos recuperar uma característica morfológica que o token **não possui**? Vamos tentar recuperar a informação de aspecto para o 22º token (índice 21):

```
# Recupera informação morfológica sobre aspecto para o Token no índice 21 do objeto Doc
doc[21].morph.get('Aspect')
```

Isso retorna uma **lista vazia**, indicada pelos colchetes `[]` sem nada entre eles.

Para recuperar **toda** a informação morfológica disponível para um dado **Token**, a melhor solução é usar o método `to_dict()` do atributo `morph`. Isso retorna um **dicionário**, uma estrutura de dados do Python composta por pares de chave e valor:

```
# Recupera informação morfológica para o Token no índice 21 do objeto Doc
# Usa o método to_dict() para converter o resultado em um dicionário
doc[21].morph.to_dict()
```

Um dicionário Python é delimitado por chaves `{ }`. Cada par chave/valor é separado por dois-pontos `:`. Nesse caso, tanto as chaves quanto os valores são objetos *string*. O valor guardado em uma chave pode ser acessado colocando o nome da chave entre colchetes `[]` logo após o nome do dicionário:

```
# Atribui a informação morfológica ao dicionário 'morph_dict'
morph_dict = doc[21].morph.to_dict()

# Recupera o valor correspondente a chave 'Mood'
morph_dict['Mood']
```

Dicionários são uma estrutura de dados poderosa em Python, que usaremos com frequência para guardar informações.

2.4 Análise sintática (*parsing* de dependências)

A **análise sintática** (ou *parsing* de dependências) é a tarefa de definir as **dependências sintáticas** entre tokens. Essas dependências ficam disponíveis no atributo `dep_` de um objeto **Token**:

```
# Percorre os itens no objeto Doc, usando a variável 'token' para se referir aos itens da
→ lista
for token in doc:

    # Imprime o token e sua etiqueta de dependência
    print(token, token.dep_)
```

Diferentemente das anotações morfossintáticas, associadas a um único **Token**, as etiquetas de dependência indicam uma **relação** entre dois tokens. Para entender melhor essas relações sintáticas, vamos usar alguns atributos adicionais de cada **Token**:

- **i**: a posição do token no **Doc**;
- **token**: o próprio token;
- **dep_**: a etiqueta da relação sintática;
- **head** e **i**: o token que **governa** o token atual, e o índice desse token-governante.

Isso ilustra como os atributos do Python podem ser combinados de forma flexível: o atributo **head** aponta para outro **Token**, que por sua vez tem o atributo **i** com seu próprio índice no **Doc**. Podemos combinar os dois atributos e acessar essa informação com **.head.i**:

```
# Percorre os itens no objeto Doc, usando a variável 'token' para se referir aos itens da
↳ lista
for token in doc:

    # Imprime o índice do token atual, o token, a dependência, o governante e seu índice
    print(token.i, token, token.dep_, token.head.i, token.head)
```

Embora a saída acima ajude a esclarecer as dependências sintáticas entre os tokens, elas costumam ser **muito mais fáceis de perceber** por meio de diagramas. O spaCy oferece uma ferramenta de visualização chamada **displaCy**, importável com o comando:

```
from spacy import displacy
```

O módulo **displacy** tem uma função chamada **render()**, que recebe um objeto **Doc** como entrada. Para desenhar uma árvore de dependências, passamos o **Doc doc** à função **render()** com dois argumentos:

- **style**: o valor **'dep'** instrui o displaCy a desenhar uma visualização de dependências sintáticas;
- **options**: recebe um dicionário Python como entrada. Passamos um dicionário com a chave **compact** e o valor booleano **True** para instruir o displaCy a desenhar uma árvore compacta.

```
displacy.render(doc, style='dep', options={'compact': True})
```

As dependências sintáticas são visualizadas por meio de **linhas** que partem do token-governante em direção ao token governado por ele. As etiquetas de dependência são baseadas no projeto **Universal Dependencies**, um arcabouço para descrever características morfológicas e sintáticas entre diferentes idiomas.

Se você não souber o significado de uma etiqueta específica, o spaCy oferece uma função para explicá-las, **explain()**, que recebe a etiqueta como entrada (atenção: as etiquetas diferenciam maiúsculas de minúsculas):

```
spacy.explain('pobj')
```

Por fim, se você está se perguntando sobre os sublinhados **_** nos nomes dos atributos: o spaCy codifica todas as **strings** mapeando-as para **valores de hash** (uma representação numérica) por eficiência computacional. Vamos imprimir o primeiro token do **Doc** (**doc[0]**) e suas dependências para examinar como isso funciona:

```
print(doc[0], doc[0].dep, doc[0].dep_)
```

Como se vê, o valor de hash **415** está reservado para a etiqueta correspondente a um determinante (**det**).

Atenção: se você quiser uma saída legível para humanos na análise de dependências e o spaCy devolver sequências de números, provavelmente você esqueceu de adicionar o sublinhado ao nome do atributo (por exemplo, usou `dep` em vez de `dep_`).

2.5 Segmentação de sentenças

O spaCy também segmenta objetos `Doc` em **sentenças** — tarefa conhecida como **segmentação de sentenças**. Essa segmentação impõe estrutura adicional a textos maiores: ao determinar os limites de uma sentença, podemos restringir tarefas como a análise de dependências a sentenças individuais.

O spaCy disponibiliza o resultado da segmentação de sentenças no atributo `sents` de um objeto `Doc`. Vamos percorrer as sentenças contidas em `doc` e contá-las usando a função `enumerate()` do Python, que retorna uma contagem crescente a cada item do laço:

```
# Percorre as sentenças no objeto Doc e as conta usando enumerate()
for number, sent in enumerate(doc.sents):

    # Imprime o número e a sentença
    print(number, sent)
```

Isso retorna apenas **uma sentença**, mas o objeto `Doc` poderia facilmente conter um texto mais longo com múltiplas sentenças, como uma reportagem inteira.

2.6 Lematização

Um **lema** é a forma base de uma palavra. Tenha em mente que, a menos que instruído explicitamente, o computador não sabe distinguir formas singular e plural de uma palavra — ele as trata como tokens distintos, porque suas formas diferem.

Se quisermos contar a **ocorrência de palavras**, por exemplo, é necessário um processo chamado **lematização**, que agrupa as diferentes formas de um mesmo token. Os lemas ficam disponíveis para cada `Token` no atributo `lemma_`:

```
# Percorre os itens no objeto Doc, usando a variável 'token' para se referir aos itens da
→ lista
for token in doc:

    # Imprime o token e seu lema
    print(token, token.lemma_)
```

2.7 Reconhecimento de entidades nomeadas (*named entity recognition*, NER)

O **reconhecimento de entidades nomeadas** (NER) é a tarefa de identificar e classificar entidades mencionadas em um texto. O spaCy reconhece as entidades nomeadas anotadas no corpus OntoNotes 5 — pessoas, locais geográficos e produtos, entre outros exemplos.

Usamos o atributo `.ents` do objeto `Doc` para obter as entidades nomeadas:

```
doc.ents
```

Isso retorna uma **tupla** com as entidades nomeadas. Cada item da tupla é um objeto `Span` do spaCy — objetos `Span` podem conter **múltiplos** objetos `Token`, já que muitas entidades nomeadas se estendem por

mais de um token. As entidades nomeadas e seus tipos ficam guardados nos atributos `.text` e `.label_` de cada `Span`. Vamos percorrer a tupla e imprimir ambos os atributos:

```
# Percorre as entidades nomeadas no objeto Doc
for ent in doc.ents:

    # Imprime a entidade nomeada e sua etiqueta
    print(ent.text, ent.label_)
```

Como se vê, a maioria das entidades nomeadas identificadas consiste em **múltiplos tokens**, por isso são representadas como objetos `Span`. Podemos confirmar isso acessando a primeira entidade nomeada (índice 0, já que o Python conta do zero) e passando o objeto à função `type()`:

```
# Verifica o tipo do objeto usado para armazenar entidades nomeadas
type(doc.ents[0])
```

Objetos `Span` do spaCy têm vários argumentos úteis. Mais importante: os atributos `start` e `end` retornam os **índices dos tokens** que determinam onde o `Span` começa e termina no `Doc`. Vamos examinar isso com mais detalhe, imprimindo os atributos `start` e `end` da primeira entidade nomeada:

```
# Imprime a entidade nomeada e os índices de seus tokens de início e fim
print(doc.ents[0], doc.ents[0].start, doc.ents[0].end)
```

A entidade nomeada começa no índice 0 e termina no índice 5 do `Doc`. Se recuperarmos o sexto token do `Doc` (índice 5), veremos que ele corresponde ao token "has":

```
doc[5]
```

Isso mostra que o índice retornado por `end` **não** corresponde ao último token do `Span` que contém a entidade nomeada, mas sim ao índice do **primeiro token seguinte** ao `Span`. Vamos examinar isso percorrendo o recorte (*slice*) do `Doc` correspondente à primeira entidade nomeada:

```
# Percorre um recorte do objeto Doc que cobre a primeira entidade nomeada
for token in doc[doc.ents[0].start: doc.ents[0].end]:

    # Imprime o token e seu índice
    print(token, token.i)
```

Como se vê, o atributo `start` indica onde o `Span` **começa**, enquanto `end` indica onde o `Span` **terminou** (ou seja, o token logo após o seu fim).

Também podemos renderizar as entidades nomeadas com o `displacy`, o mesmo módulo usado acima para visualizar as dependências sintáticas. Note que, desta vez, passamos a *string* 'ent' ao argumento `style` para indicar que queremos visualizar **entidades nomeadas**:

```
displacy.render(doc, style='ent')
```

Se você não reconhecer uma etiqueta usada para uma entidade nomeada, pode sempre pedir uma explicação ao spaCy:

```
spacy.explain('NORP')
```


Quiz

Marque a alternativa correta (a resposta certa está destacada com ✓).

1. A tarefa que quebra o texto em unidades menores chama-se:

1. Tokenização ✓
2. Lematização
3. Segmentação de sentenças

2. Qual atributo do token dá a classe gramatical genérica?

1. `pos_` ✓
2. `tag_`
3. `dep_`

3. O lema de uma palavra é:

1. A forma base da palavra ✓
2. Sempre o plural
3. A raiz fonética

4. As entidades nomeadas de um `Doc` ficam em qual atributo?

1. `.ents` ✓
2. `.sents`
3. `.noun_chunks`

5. Por que muitos atributos têm uma versão terminada em sublinhado (ex.: `dep_`)?

1. Ela dá o texto legível, em vez do código numérico (hash) ✓
2. Ela é mais rápida
3. Ela é obrigatória

6. Uma entidade nomeada com vários tokens é representada por um objeto:

1. `Span` ✓
2. `Token`
3. `Doc`

Resumo da unidade

Nesta unidade, você aprendeu a:

1. **carregar** um modelo de linguagem com `spacy.load()` e entender o que é e como é treinado um modelo estatístico de linguagem;
2. **processar** um texto com o objeto `Language(nlp(text))`, obtendo um objeto `Doc`;
3. **tokenizar** o texto e percorrer os objetos `Token` resultantes;
4. **anotar classes gramaticais** (`pos_`, `tag_`) e realizar **análise morfológica** (`morph`, `to_dict()`);
5. **analisar dependências sintáticas** (`dep_`, `head`), visualizando-as com `displacy.render(..., style='dep')`;
6. **segmentar sentenças** (`doc.sents`) e **lematizar** tokens (`lemma_`);
7. **reconhecer entidades nomeadas** (`doc.ents`, `.text`, `.label_`), entendendo os atributos `start/end` de um `Span`, e visualizá-las com `displacy.render(..., style='ent')`.

Exercícios sugeridos

1. Processe uma das notícias completas da Unidade 1 com `nlp()` e conte quantas sentenças o `Doc` resultante contém.
 2. Liste todos os tokens classificados como verbo (`token.pos_ == 'VERB'`) em um texto à sua escolha.
 3. Extraia todas as entidades nomeadas do tipo pessoa (`PERSON`) de um texto e imprima seus lemas.
 4. Use `spacy.explain()` para descobrir o significado de pelo menos três etiquetas de dependência (`dep_`) diferentes das usadas nos exemplos acima.
-

Referências

- Rognvaldsson, E. et al. *Applied Language Technology* (MOOC). Universidade de Helsinque. <https://applied-language-technology.mooc.fi/>
- de Marneffe, M.-C. et al. (2021). Universal Dependencies. *Computational Linguistics*.
- Honnibal, M.; Montani, I. et al. *spaCy: Industrial-strength Natural Language Processing in Python*. <https://spacy.io/>

Customizando o pipeline do spaCy

Trabalhando com Texto em Python I · Unidade 3

CiberExt 26-29 · FEELT38103 · Universidade Federal de Uberlândia

2026

Sumário

1. Modificando pipelines do spaCy	1
2. Processando textos com eficiência	3
3. Adicionando atributos personalizados aos objetos do spaCy	4
4. Gravando textos processados em disco	6
5. Simplificando a saída: sintagmas nominais e entidades nomeadas	8
Quiz	11
Resumo da unidade	11

Objetivos de aprendizagem

Ao final desta unidade, você deverá saber:

- **examinar e modificar** o pipeline do spaCy;
- **processar textos com eficiência**;
- **adicionar atributos personalizados** a objetos do spaCy;
- **salvar em disco** os textos processados;
- **mesclar** sintagmas nominais e entidades nomeadas em tokens únicos.

Vamos começar importando a biblioteca spaCy e o módulo `displacy`, usado para desenhar árvores de dependência, e carregando um modelo de linguagem para o inglês:

```
# Importa a biblioteca spaCy e o módulo displacy
from spacy import displacy
import spacy

# Carrega um modelo pequeno de língua inglesa e o atribui a variável 'nlp'
nlp = spacy.load('en_core_web_sm')

# Chama a variável para examinar o objeto
nlp
```

```
<spacy.lang.en.English at 0x294522f10>
```

1. Modificando pipelines do spaCy

O objeto `Language` é, essencialmente, um **pipeline** que aplica um modelo de linguagem ao texto, executando as tarefas para as quais o modelo foi treinado. As tarefas realizadas dependem dos **componentes** presentes no pipeline.

Podemos examinar os componentes de um pipeline usando o atributo `pipeline` de um objeto `Language`:

```
nlp.pipeline
```

```
[('tok2vec', <spacy.pipeline.tok2vec.Tok2Vec at 0x294cf3dc0>),  
(('tagger', <spacy.pipeline.tagger.Tagger at 0x294cf3ca0>),  
(('parser', <spacy.pipeline.dep_parser.DependencyParser at 0x294b566d0>),  
(('attribute_ruler', <spacy.pipeline.attributeruler.AttributeRuler at 0x295187680>),  
(('lemmatizer', <spacy.lang.en.lemmatizer.EnglishLemmatizer at 0x295191540>),  
(('ner', <spacy.pipeline.ner.EntityRecognizer at 0x294b56740>))]]
```

Isso retorna um objeto `SimpleFrozenList` do spaCy, composto por **tuplas** do Python com dois itens:

- o **nome** do componente (por exemplo, `tagger`);
- o **componente** que efetivamente executa a tarefa (por exemplo, `spacy.pipeline.tok2vec.Tok2Vec`).

Componentes como `tagger`, `parser`, `ner` e `lemmatizer` já devem ser familiares da unidade anterior. Há, porém, dois componentes que ainda não encontramos:

- `tok2vec` mapeia os tokens para suas representações numéricas (veremos essas representações na Parte III).
- `attribute_ruler` aplica regras definidas pelo usuário aos tokens — por exemplo, correspondências a um dado padrão linguístico — e adiciona essa informação ao token como um atributo, se solicitado.

Nota: a lista de componentes em `nlp.pipeline` **não** inclui o `Tokenizer`, porque todo texto precisa ser tokenizado para que qualquer processamento ocorra. Por isso, o `Tokenizer` fica no atributo `tokenizer` do objeto `Language`, e não no atributo `pipeline`.

1.1 Excluindo componentes para ganhar desempenho

É importante entender que **todo componente do pipeline tem um custo computacional**. Se você não precisa da saída de um componente, não deve incluí-lo no pipeline, pois o processamento ficará mais lento.

Para excluir um componente, forneça o argumento `exclude` com uma *string* (ou uma lista) contendo os nomes dos componentes a excluir, ao inicializar o objeto `Language` com a função `load()`:

```
# Carrega um modelo pequeno de língua inglesa, mas exclui o reconhecimento de  
# entidades nomeadas ('ner') e a análise de dependências sintáticas ('parser').  
nlp = spacy.load('en_core_web_sm', exclude=['ner', 'parser'])  
  
# Examina os componentes ativos no objeto Language 'nlp'  
nlp.pipeline
```

```
[('tok2vec', <spacy.pipeline.tok2vec.Tok2Vec at 0x29514d760>),  
(('tagger', <spacy.pipeline.tagger.Tagger at 0x296c7ba00>),  
(('attribute_ruler', <spacy.pipeline.attributeruler.AttributeRuler at 0x296c6bb40>),  
(('lemmatizer', <spacy.lang.en.lemmatizer.EnglishLemmatizer at 0x296c72200>))]]
```

Como mostra a saída, os componentes `ner` e `parser` **não** estão mais no pipeline.

1.2 Analisando o pipeline com `analyze_pipes()`

O objeto `Language` também fornece o método `analyze_pipes()`, que dá uma visão geral dos componentes e de suas interações. Definindo o atributo `pretty` como `True`, o `spaCy` imprime uma tabela com os componentes e as anotações que produzem:

```
# Analisa o pipeline e guarda a análise na variável 'pipe_analysis'
pipe_analysis = nlp.analyze_pipes(pretty=True)
```

```
===== Pipeline Overview =====

#   Component      Assigns      Requires      Scores      Retokenizes
-   -
0   tok2vec        doc.tensor
1   tagger         token.tag      tag_acc      False
2   attribute_ruler
3   lemmatizer     token.lemma    lemma_acc    False

✓ No problems found.
```

O método `analyze_pipes()` retorna um **dicionário** Python, com a mesma informação da tabela. Você pode usá-lo para verificar que nenhum problema foi encontrado **antes** de processar grandes volumes de dados. Os relatórios de problemas ficam guardados sob a chave `problems`:

```
# Examina o valor guardado sob a chave 'problems'
pipe_analysis['problems']
```

```
{'tok2vec': [], 'tagger': [], 'attribute_ruler': [], 'lemmatizer': []}
```

Isso retorna um dicionário com os nomes dos componentes como chaves, cujos valores são **listas de problemas**. Neste caso, as listas estão vazias, porque não há problemas.

Podemos escrever facilmente um trecho de código que **verifica** se isso é realmente verdade. Para isso, percorremos o dicionário `pipe_analysis` usando o método `items()` para obter os pares chave/valor, e usamos a instrução `assert` com a função `len()` e o operador de comparação `==` para checar que a lista tem tamanho `0`. Se essa afirmação **não** for verdadeira (ou seja, se houver algum problema), o Python levanta um `AssertionError` e para:

```
# Percorre os pares chave/valor do dicionário. Atribui a chave e o valor
# as variáveis 'component_name' e 'problem_list'.
for component_name, problem_list in pipe_analysis['problems'].items():

    # Usa 'assert' para checar a lista de problemas; levanta Error se necessário.
    assert len(problem_list) == 0, f"There is a problem with {component_name}:
    {problem_list}!"
```

Aqui também imprimimos uma mensagem de erro usando uma *f-string*: o `f` antes das aspas declara que a *string* pode ser formatada, permitindo **inserir variáveis** entre chaves `{}`. Se um erro for levantado, essas partes serão preenchidas com os valores atuais de `component_name` e `problem_list`. Se nenhum problema for encontrado, o laço passa silenciosamente.

2. Processando textos com eficiência

Ao trabalhar com grandes volumes de dados, é altamente desejável processá-los da forma mais eficiente possível. Para ilustrar as boas práticas, vamos definir um exemplo com uma **lista** de três frases da Wikipédia em inglês:

```
# Reinicializa o modelo de linguagem, pois precisamos da análise de
# dependências nas próximas seções.
nlp = spacy.load('en_core_web_sm')

# Define uma lista de frases de exemplo
sents = ["On October 1, 2009, the Obama administration went ahead with a Bush
→ administration program, increasing nuclear weapons production.",
        "The 'Complex Modernization' initiative expanded two existing nuclear sites to
→ produce new bomb parts.",
        "The administration built new plutonium pits at the Los Alamos lab in New Mexico
→ and expanded enriched uranium processing at the Y-12 facility in Oak Ridge,
→ Tennessee."]

# Chama a variável para examinar a saída
sents
```

Os objetos **Language** do spaCy têm um método específico, **pipe()**, para processar textos guardados em uma **lista**. Esse método foi otimizado para essa finalidade: ele processa os textos em **lotes** (*batches*), em vez de individualmente, o que o torna mais rápido do que processar cada item com um laço **for**.

O método **pipe()** recebe uma lista como entrada e retorna um **gerador** (*generator*) do Python:

```
# Alimenta a lista de frases ao método pipe()
docs = nlp.pipe(sents)

# Chama a variável para examinar a saída
docs
```

```
<generator object Language.pipe at 0x296d0b580>
```

Geradores são objetos do Python que contêm outros objetos. Quando chamado, um gerador **produz** (*yields*) os objetos contidos nele. Para recuperar todos os objetos de um gerador, precisamos convertê-lo em outro tipo, como uma **lista** — pense na lista como uma estrutura capaz de coletar a saída do gerador para exame:

```
# Converte o gerador pipe em uma lista
docs = list(docs)

# Chama a variável para examinar a saída
docs
```

Isso nos dá uma lista de objetos **Doc** do spaCy prontos para processamento posterior.

3. Adicionando atributos personalizados aos objetos do spaCy

A unidade anterior mostrou como acessar as anotações linguísticas do spaCy por meio de seus atributos. Além disso, o spaCy permite **definir atributos personalizados** para objetos **Doc**, **Span** e **Token**. Esses atributos podem guardar, por exemplo, informações adicionais sobre os textos — se você trabalha com textos

que trazem dados sobre os usuários da língua, pode incorporar essa informação diretamente nos objetos do spaCy.

Atributos personalizados são adicionados ao objeto `Doc` com o método `set_extension()`. Como esses atributos são adicionados a **todos** os objetos `Doc` (e não a um `Doc` individual), primeiro importamos o objeto `Doc` genérico do módulo `tokens` do spaCy:

```
# Importa o objeto Doc do módulo 'tokens' do spaCy
from spacy.tokens import Doc

# Adiciona dois atributos personalizados ao objeto Doc, 'age' e 'location',
# usando o método set_extension().
Doc.set_extension("age", default=None)
Doc.set_extension("location", default=None)
```

Usamos o argumento `default` para definir um valor padrão para ambos, com a palavra-chave `None` do Python.

Nota: diferentemente de atributos como `sents` ou `heads`, os atributos personalizados ficam sob um atributo que consiste no caractere sublinhado `_` — por exemplo, `Doc._.age`.

Para exemplificar, vamos definir um dicionário Python. O dicionário `sents_dict` tem três chaves (0, 1 e 2), cujos valores são, por sua vez, dicionários com três chaves: `age`, `location` e `text`. Isso mostra como as estruturas de dados do Python costumam ser **aninhadas**:

```
# Cria um dicionário cujos valores são outros dicionários
# com três chaves: 'age', 'location' e 'text'.
sents_dict = {0: {"age": 23,
                  "location": "Helsinki",
                  "text": "The Senate Square is by far the most important landmark in
↪ Helsinki."},
              1: {"age": 35,
                  "location": "Tallinn",
                  "text": "The Old Town, for sure."},
              2: {"age": 58,
                  "location": "Stockholm",
                  "text": "Södermalm is interesting!"},
              }
```

Vamos percorrer o dicionário `sents_dict` para processar os exemplos e adicionar os atributos personalizados aos objetos `Doc` resultantes:

```
# Prepara uma lista vazia para guardar os textos processados
docs = []

# Percorre os pares de chave e valor do dicionário 'sents_dict'.
# Os pares chave/valor ficam disponíveis pelo método items().
# Chamamos essas chaves e valores de 'key' e 'data'; ou seja, usamos
# a variável 'data' para se referir ao dicionário aninhado.
for key, data in sents_dict.items():
```

```

# Recupera o valor da chave 'text' do dicionário aninhado.
# Alimenta esse texto ao modelo em 'nlp' e guarda o resultado em 'doc'.
doc = nlp(data['text'])

# Recupera os valores de 'age' e 'location' do dicionário aninhado.
# Atribui esses valores aos atributos personalizados do objeto Doc.
# Lembre-se: atributos personalizados ficam sob o pseudo-atributo '_'!
doc._.age = data['age']
doc._.location = data['location']

# Acrescenta o Doc atual em 'doc' a lista 'docs'
docs.append(doc)

```

Isso produz uma lista de objetos `Doc`, guardada em `docs`. Vamos percorrê-la e imprimir cada `Doc` com seus atributos personalizados:

```

# Percorre cada objeto Doc na lista 'docs'
for doc in docs:

    # Imprime cada Doc e os atributos 'age' e 'location'
    print(doc, doc._.age, doc._.location)

```

```

The Senate Square is by far the most important landmark in Helsinki. 23 Helsinki
The Old Town, for sure. 35 Tallinn
Södermalm is interesting! 58 Stockholm

```

3.1 Filtrando dados com compreensão de lista

Os atributos personalizados podem ser usados, por exemplo, para **filtrar** os dados. Uma forma eficiente é a **compreensão de lista** (*list comprehension*), que avalia o conteúdo de uma lista existente e monta uma nova lista com base em algum critério — é como um laço `for` declarado “na hora”, entre colchetes `[]`. Ela tem três componentes:

1. a referência a `doc` à esquerda do `for` define o que será guardado na nova lista (no caso, o próprio objeto `Doc`);
2. o `for ... in` funciona como num laço comum, percorrendo os itens de `docs`;
3. o `if` define uma **condição**: só incluímos objetos `Doc` cujo atributo `age` tenha valor abaixo de 40.

```

# Usa uma compreensao de lista para filtrar os Docs cujo atributo
# 'age' tenha valor abaixo de 40.
under_forty = [doc for doc in docs if doc._.get('age') < 40]

# Chama a variável para examinar a saída
under_forty

```

```

[The Senate Square is by far the most important landmark in Helsinki.,
The Old Town, for sure.]

```

Isso retorna uma lista com apenas dois objetos `Doc` que atendem ao critério.

4. Gravando textos processados em disco

Ao trabalhar com grandes volumes de textos, primeiro garanta que o pipeline produz o resultado desejado usando **poucos** textos. Quando tudo estiver funcionando, processe o conjunto completo e **salve o resultado**, pois processar grandes volumes consome tempo e recursos.

O spaCy oferece um tipo de objeto especial, o `DocBin`, para armazenar objetos `Doc` com suas anotações linguísticas:

```
# Importa o objeto DocBin do módulo 'tokens' do spaCy
from spacy.tokens import DocBin

# Inicializa um objeto DocBin e adiciona os Docs de 'docs'
docbin = DocBin(docs=docs)
```

Atenção: se você adicionou **atributos personalizados** a `Docs`, `Spans` ou `Tokens`, também é preciso definir o argumento `store_user_data` como `True` — por exemplo, `DocBin(docs=docs, store_user_data=True)`.

Podemos verificar que os três `Docs` entraram no `DocBin` examinando a saída do método `__len__()`:

```
# Obtem o número de Docs no DocBin
docbin.__len__()
```

```
3
```

O método `add()` permite adicionar mais objetos `Doc` ao `DocBin`, se necessário:

```
# Define uma string, alimenta o modelo em 'nlp' e adiciona o
# Doc resultante ao objeto DocBin 'docbin'
docbin.add(nlp("Yet another Doc object."))

# Verifica que o Doc foi adicionado; o tamanho agora deve ser 4
docbin.__len__()
```

```
4
```

Depois de populado, o `DocBin` deve ser **gravado em disco** com o método `to_disk()`, que recebe um único argumento, `path`, definindo o caminho do arquivo. Vamos gravar o `DocBin` em um arquivo chamado `docbin.spacy`, no diretório `data`:

```
# Grava o objeto DocBin em disco
docbin.to_disk(path='data/docbin.spacy')
```

Para **carregar** um `DocBin` do disco, primeiro inicialize um `DocBin` vazio com `DocBin()` e use o método `from_disk()`:

```
# Inicializa um novo DocBin e usa 'from_disk' para carregar os dados do disco.
# Atribui o resultado a variável 'docbin_loaded'.
docbin_loaded = DocBin().from_disk(path='data/docbin.spacy')

# Chama a variável para examinar a saída
docbin_loaded
```

```
<spacy.tokens._serialize.DocBin at 0x2953a4c40>
```

Por fim, para acessar os objetos `Doc` guardados no `DocBin`, use o método `get_docs()`. Ele recebe um único argumento, `vocab`, que precisa do **vocabulário** de um objeto `Language` (guardado no atributo `vocab`) para reconstruir a informação armazenada no `DocBin`:

```
# Usa 'get_docs' para recuperar os Docs do DocBin, passando o
# vocabulário em 'nlp.vocab' para reconstruir os dados.
# Converte o gerador resultante em uma lista para exame.
docs_loaded = list(docbin_loaded.get_docs(nlp.vocab))

# Chama a variável para examinar a saída
docs_loaded
```

```
[The Senate Square is by far the most important landmark in Helsinki.,
The Old Town, for sure.,
Södermalm is interesting!,
Yet another Doc object.]
```

Isso retorna uma lista com os quatro `Docs` adicionados ao `DocBin`.

Resumindo: o ideal é processar os textos **uma vez**, gravá-los em disco e carregá-los para as análises seguintes.

5. Simplificando a saída: sintagmas nominais e entidades nomeadas

5.1 Mesclando sintagmas nominais

Tarefas como anotação morfosintática e análise de dependências fazem previsões sobre **tokens individuais**. Às vezes, porém, é mais vantajoso operar com unidades linguísticas maiores, como **sintagmas nominais** (*noun phrases*) formados por vários tokens.

O spaCy dá acesso aos sintagmas nominais pelo atributo `noun_chunks` de um objeto `Doc`. Vamos imprimir os sintagmas de cada `Doc` na lista `docs`:

```
# Primeiro laço: percorre a lista 'docs'; 'doc' se refere aos itens da lista
for doc in docs:

    # Percorre cada sintagma nominal do objeto Doc
    for noun_chunk in doc.noun_chunks:

        # Imprime o sintagma nominal
        print(noun_chunk)
```

```
The Senate Square
the most important landmark
Helsinki
The Old Town
Södermalm
```

Para **mesclar** os sintagmas nominais em um único token, o spaCy oferece a função `merge_noun_chunks`, que pode ser adicionada ao pipeline com o método `add_pipe`:

```
# Adiciona o componente que mescla sintagmas nominais em Tokens únicos
nlp.add_pipe('merge_noun_chunks')
```

Nota: não é preciso reatribuir o objeto `Language` à mesma variável para atualizá-lo — o método `add_pipe` adiciona o componente automaticamente.

Processando novamente as três frases com `pipe()`, tudo **parece** igual (uma lista com três `Docs`). Mas, ao percorrer os tokens do primeiro `Doc` (`[0]`), vemos que os sintagmas nominais agora estão **mesclados** e rotulados como `NOUN`:

```
# Aplica o objeto Language 'nlp' a lista de frases em 'sents'
docs = list(nlp.pipe(sents))

# Percorre os Tokens do primeiro Doc da lista
for token in docs[0]:

    # Imprime o Token e sua classe gramatical
    print(token, token.pos_)
```

```
On ADP
October PROPN
1 NUM
, PUNCT
2009 NUM
, PUNCT
the Obama administration NOUN
went VERB
ahead ADV
with ADP
a Bush administration program NOUN
, PUNCT
increasing VERB
nuclear weapons production NOUN
. PUNCT
```

Rotular os sintagmas nominais como `NOUN` é uma aproximação razoável, já que suas palavras-núcleo são substantivos. Como a renderização com `displaCy` mostra, mesclar os sintagmas **simplifica** a árvore sintática:

```
displacy.render(docs[0], style='dep')
```

Embora os sintagmas nominais estejam agora representados por tokens únicos, eles continuam disponíveis no atributo `noun_chunks`. O `spaCy` os guarda como objetos `Span`, cujos atributos `start` e `end` determinam onde o `Span` começa e termina:

```
# Percorre os sintagmas nominais do primeiro Doc [0] na lista 'docs'
for noun_chunk in docs[0].noun_chunks:

    # Imprime o sintagma, seu tipo, e os índices onde começa e termina
    print(noun_chunk, type(noun_chunk), noun_chunk.start, noun_chunk.end)
```

```
October <class 'spacy.tokens.span.Span'> 1 2
the Obama administration <class 'spacy.tokens.span.Span'> 6 7
a Bush administration program <class 'spacy.tokens.span.Span'> 10 11
```

```
nuclear weapons production <class 'spacy.tokens.span.Span'> 13 14
```

5.2 Mesclando entidades nomeadas

Entidades nomeadas podem ser mescladas do mesmo modo, fornecendo `merge_entities` ao método `add_pipe()`. Primeiro, removemos a função `merge_noun_chunks` do pipeline com o método `remove_pipe()`:

```
# Remove a função 'merge_noun_chunks' do pipeline em 'nlp'
nlp.remove_pipe('merge_noun_chunks')

# Processa as frases originais novamente
docs = list(nlp.pipe(sents))
```

O método retorna uma tupla com o nome do componente removido e o próprio componente. Em seguida, adicionamos o componente `merge_entities`:

```
# Adiciona a função 'merge_entities' ao pipeline
nlp.add_pipe('merge_entities')

# Processa os dados novamente
docs = list(nlp.pipe(sents))

# Percorre os Tokens do terceiro Doc da lista
for token in docs[2]:

    # Imprime o Token e sua classe gramatical
    print(token, token.pos_)
```

```
The DET
administration NOUN
built VERB
new ADJ
plutonium NOUN
pits NOUN
at ADP
the DET
Los Alamos PROPN
lab NOUN
in ADP
New Mexico PROPN
and CCONJ
expanded VERB
enriched ADJ
uranium NOUN
processing NOUN
at ADP
the DET
Y-12 NUM
facility NOUN
in ADP
Oak Ridge PROPN
, PUNCT
Tennessee PROPN
```

Entidades nomeadas com vários tokens — como os topônimos “*Los Alamos*” e “*New Mexico*” — foram **mescladas** em tokens únicos.

Quiz

Marque a alternativa correta (a resposta certa está destacada com ✓).

1. Por que remover do pipeline componentes que você não usa?

1. Cada componente tem um custo computacional ✓
2. Deixa o código mais bonito
3. É obrigatório

2. Qual argumento de `load()` exclui componentes do pipeline?

1. `exclude` ✓
2. `remove`
3. `skip`

3. Por que o `Tokenizer` não aparece em `nlp.pipeline`?

1. Ele sempre roda e fica em `nlp.tokenizer` ✓
2. Foi removido do spaCy
3. É opcional e estava desligado

4. Onde ficam os atributos personalizados de um `Doc`?

1. Sob um atributo que consiste no sublinhado (`Doc._`) ✓
2. Numa coluna extra do texto
3. Dentro de `nlp.meta`

5. Qual método processa uma lista de textos em lote (mais rápido)?

1. `pipe()` ✓
2. `apply()`
3. `map()`

6. Para que serve o `DocBin`?

1. Guardar e gravar `Docs` processados em disco ✓
2. Baixar modelos de linguagem
3. Treinar o modelo

Resumo da unidade

Nesta unidade, você aprendeu a:

1. **examinar** o pipeline com `nlp.pipeline` e entender componentes como `tok2vec` e `attribute_ruler`;
2. **excluir componentes** (`exclude=[...]`) para ganhar desempenho e **auditar** o pipeline com `analyze_pipes()` e `assert`;
3. **processar textos em lote** com `nlp.pipe()`, entendendo geradores e a conversão para lista;

4. **adicionar atributos personalizados** (`set_extension()`, `Doc._.`) e **filtrar** dados com compreensão de lista;
5. **gravar e carregar** textos processados com `DocBin` (`to_disk()`, `from_disk()`, `get_docs()`);
6. **mesclar** sintagmas nominais (`merge_noun_chunks`) e entidades nomeadas (`merge_entities`) com `add_pipe()/remove_pipe()`.

Exercícios sugeridos

1. Carregue o modelo excluindo apenas o `lemmatizer` e confirme, com `nlp.pipeline`, que ele não está mais presente.
 2. Adicione um atributo personalizado `source` ao `Doc` e preencha-o com o nome do arquivo de origem ao processar textos da Unidade 1.
 3. Processe as notícias da Unidade 1 com `nlp.pipe()`, grave tudo em um `DocBin` e recarregue do disco, conferindo o número de `Docs`.
 4. Compare a árvore de dependências (`displacy`) de uma frase **com** e **sem** `merge_noun_chunks` e descreva a diferença.
-

Referências

- Rognvaldsson, E. et al. *Applied Language Technology* (MOOC). Universidade de Helsinque. <https://applied-language-technology.mooc.fi/>
- Honnibal, M.; Montani, I. et al. *spaCy: Industrial-strength Natural Language Processing in Python*. <https://spacy.io/>

Avaliando modelos de linguagem

Trabalhando com Texto em Python I · Unidade 4

CiberExt 26-29 · FEELT38103 · Universidade Federal de Uberlândia

2026

Sumário

1. O que é um padrão-ouro (<i>gold standard</i>)?	1
2. Medindo a confiabilidade manualmente	1
3. O kappa de Cohen como medida de concordância	4
4. Avaliando o desempenho de modelos de linguagem	5
Quiz	9
Resumo da unidade	9

Objetivos de aprendizagem

Ao final desta unidade, você deverá:

- entender o que é um **padrão-ouro** (*gold standard*);
- saber **avaliar a concordância** entre anotadores humanos;
- compreender **métricas simples** para avaliar o desempenho do processamento de linguagem natural.

1. O que é um padrão-ouro (*gold standard*)?

Um **padrão-ouro** (*gold standard*) — também chamado de *ground truth* (“verdade fundamental”) — refere-se a **dados verificados por humanos** que podem ser usados como referência (*benchmark*) para avaliar o desempenho de algoritmos. Em processamento de linguagem natural, os padrões-ouro medem quão bem os **humanos** desempenham uma tarefa.

O objetivo do PLN é permitir que computadores **alcancem ou superem** o desempenho humano em uma tarefa pré-definida. Medir se os algoritmos conseguem isso exige uma referência — e é o padrão-ouro que a fornece. Em termos simples, ele oferece um **ponto de referência**.

É importante entender, porém, que **padrões-ouro são abstrações do uso da língua**. Considere a tarefa de classificar palavras em classes gramaticais: as classes não são dadas pela natureza — elas representam uma abstração que impõe estrutura à língua. A língua, entretanto, é naturalmente **ambígua e subjetiva**, e as abstrações usadas podem ser incompletas: não temos certeza de que todos os usuários classificariam as palavras da mesma forma.

Por isso precisamos **medir a confiabilidade** de qualquer padrão-ouro, ou seja, até que ponto os humanos **concordam** na tarefa.

2. Medindo a confiabilidade manualmente

Esta seção mostra como a **confiabilidade** — geralmente entendida como a **concordância entre múltiplos anotadores** — pode ser medida manualmente.

Passo 1 — Anotar os dados

A **análise de sentimento** é uma tarefa que consiste em determinar o sentimento de um texto. Treinar um modelo de análise de sentimento exige coletar dados de treino, isto é, exemplos de textos associados a diferentes sentimentos.

Classifique os *tweets* a seguir em três categorias — **positivo**, **neutro** ou **negativo** — de acordo com o sentimento. Anote sua decisão (uma por linha), mas **não** discuta nem mostre ao colega ao lado:

1. Updated: HSL GTFS (Helsinki, Finland) <https://t.co/fWEpzmNQLz>
2. current weather in Helsinki: broken clouds, -8°C 100% humidity, wind 4kmh, pressure 1061mb
3. CNN: “WallStreetBets Redditors go ballistic over GameStop’s sinking share price”
4. Baana bicycle counter. Today: 3 Same time last week: 1058 Trend: ↓99% This year: 819 518 Last year: 802 079 #Helsinki #cycling
5. Elon Musk is now tweeting about #bitcoin
6. A perfect Sunday walk in the woods just a few steps from home.
7. Went to Domino’s today It was so amazing and I think I got damn good dessert as well. . .
8. Choo Choo There’s our train! #holidayahead
9. Happy women’s day kisses to all you beautiful ladies. #awesometobeawoman
10. Good morning #Helsinki! Sun will rise in 30 minutes (local time 07:28)

Anote suas classificações (uma por linha, de 1 a 10). O colega ao lado faz o mesmo, de forma independente — assim teremos **dois anotadores** para comparar.

Passo 2 — Calcular a concordância percentual

Ao criar conjuntos de dados para treinar modelos, geralmente queremos que os dados sejam **confiáveis**, ou seja, que haja concordância sobre o que descrevemos (aqui, o sentimento dos *tweets*).

Uma forma de medir isso é a **concordância percentual** simples: quantas vezes, das 10, você e o colega concordaram. Basta dividir o número de concordâncias pelo número de itens (10):

```
# Substitua o número abaixo pelo número de itens em que vocês concordaram
agreement = 0

# Divide a contagem pelo número de tweets
agreement = agreement / 10

# Imprime a variável
agreement
```

```
0.0
```

Passo 3 — Calcular as probabilidades de cada categoria

A concordância percentual é, na verdade, uma medida **muito ruim**: qualquer um dos dois pode ter **acertado por sorte** — ou ter achado a tarefa tediosa e classificado tudo aleatoriamente. Se isso aconteceu, a concordância percentual **não tem como detectar**, pois ela não distingue acerto real de acaso!

Felizmente, dá para estimar a possibilidade de **concordância por acaso**. O primeiro passo é contar quantas vezes **você** usou cada categoria e converter essas contagens em **probabilidades**, dividindo pelo total de *tweets*:

```
# Conte quantos itens *voce* colocou em cada categoria
positive = 0
neutral = 0
negative = 0

# Converte as contagens em probabilidades
positive = positive / 10
neutral = neutral / 10
negative = negative / 10

# Chama cada variável para examinar a saída
positive, neutral, negative
```

```
(0.0, 0.0, 0.0)
```

Essas probabilidades representam a chance de **você** escolher aquela categoria. Agora pergunte ao colega as probabilidades dele e registre-as:

```
nb_positive = 0
nb_neutral = 0
nb_negative = 0
```

Conhecendo as probabilidades de cada classe para os **dois** anotadores, podemos calcular a probabilidade de ambos escolherem a mesma categoria **por acaso**. Para cada categoria, basta multiplicar a sua probabilidade pela do colega:

```
both_positive = positive * nb_positive
both_neutral = neutral * nb_neutral
both_negative = negative * nb_negative
```

Nota: se um anotador não colocou nenhum *tweet* em uma categoria (por exemplo, *negative*) e o outro colocou, isso **anula** a chance de concordância por acaso naquela categoria — multiplicar por zero resulta em zero.

Passo 4 — Estimar a concordância esperada (por acaso)

Agora podemos calcular quão provável é concordar **por acaso**. Isso é a **concordância esperada** (*expected agreement*), obtida somando as probabilidades combinadas de cada categoria:

```
expected_agreement = both_positive + both_neutral + both_negative

expected_agreement
```

```
0.0
```

Conhecendo tanto a **concordância observada** (*agreement*) quanto a **esperada por acaso** (*expected_agreement*), podemos usar uma medida mais confiável: o **kappa de Cohen** (κ), que estima a concordância a partir de ambas. A fórmula é:

$$\kappa = \frac{P_{\text{observado}} - P_{\text{esperado}}}{1 - P_{\text{esperado}}}$$

Como essa informação já está nas variáveis `agreement` e `expected_agreement`, calculamos κ facilmente. Note que envolvemos as subtrações em **parênteses** para executá-las antes da divisão:

```
kappa = (agreement - expected_agreement) / (1 - expected_agreement)

kappa

0.0
```

3. O kappa de Cohen como medida de concordância

O valor teórico do κ de Cohen vai de **-1** (discordância perfeita) a **+1** (concordância perfeita), com **0** indicando concordância totalmente aleatória. O κ costuma ser interpretado como uma medida da **força** da concordância.

Landis e Koch (1977) propuseram os *benchmarks* abaixo — que devem ser levados **com cautela**, pois as divisões são arbitrárias:

Kappa de Cohen (κ)	Força da concordância
< 0,00	Pobre
0,00 – 0,20	Ligeira
0,21 – 0,40	Razoável
0,41 – 0,60	Moderada
0,61 – 0,80	Substancial
0,81 – 1,00	Quase perfeita

O κ de Cohen serve para medir a concordância entre **dois** anotadores, e as categorias disponíveis devem ser fixadas de antemão. Para mais de dois anotadores, usa-se uma medida como o κ **de Fleiss**.

Na prática, o κ de Cohen (e muitas outras medidas) já está implementado em bibliotecas Python — raramente é preciso calcular à mão. A biblioteca **scikit-learn** (`sklearn`), por exemplo, inclui a função `cohen_kappa_score()`, que recebe **duas listas** e calcula o κ entre elas:

```
# Importa a função cohen_kappa_score do módulo 'metrics' da scikit-learn
from sklearn.metrics import cohen_kappa_score

# Define duas listas, 'a1' e 'a2', com anotações de classes gramaticais
a1 = ['ADJ', 'AUX', 'NOUN', 'VERB', 'VERB']
a2 = ['ADJ', 'VERB', 'NOUN', 'NOUN', 'VERB']

# Usa cohen_kappa_score() para calcular a concordância entre as listas
cohen_kappa_score(a1, a2)

0.44444444444444453
```

Segundo o *benchmark* de Landis e Koch, esse valor indicaria concordância **moderada**.

Dica: raramente é preciso anotar o conjunto inteiro para medir a concordância — uma **amostra aleatória** costuma bastar. Se o κ sugere que os anotadores concordam, assumimos que as anotações são confiáveis (não aleatórias). Ainda assim, toda medida de concordância entre anotadores depende de suas próprias suposições sobre o que é “concordar” — nenhuma representa a verdade absoluta.

4. Avaliando o desempenho de modelos de linguagem

Com um padrão-ouro suficientemente confiável, podemos usá-lo para medir o desempenho de modelos de linguagem. Suponha um padrão-ouro com 10 tokens anotados por classe gramatical, na lista `gold_standard`, e as previsões de um modelo na lista `predictions`:

```
# Define a lista 'gold_standard' (padrão-ouro anotado por humanos)
gold_standard = ['ADJ', 'ADJ', 'AUX', 'VERB', 'AUX', 'NOUN', 'NOUN', 'ADJ', 'DET', 'PRON']

# Define a lista 'predictions' (previsões do modelo de linguagem)
predictions = ['NOUN', 'ADJ', 'AUX', 'VERB', 'AUX', 'NOUN', 'VERB', 'ADJ', 'DET', 'PROPN']
```

4.1 Acurácia

Vamos importar o módulo `metrics` inteiro da scikit-learn. A função `accuracy_score()` calcula a **acurácia**, que é exatamente a mesma coisa que a concordância observada calculada manualmente:

```
# Importa o módulo 'metrics' da biblioteca scikit-learn (sklearn)
from sklearn import metrics

# Usa a função accuracy_score() do módulo 'metrics'
metrics.accuracy_score(gold_standard, predictions)
```

0.7

A acurácia sofre do mesmo problema da concordância observada — pode ser resultado de **palpites de sorte**. Como é um exemplo pequeno, dá para verificar que **7 de 10** classes gramaticais coincidem, resultando em acurácia de 0,7 (70%).

4.2 Matriz de confusão

Para avaliar melhor o modelo, organizamos os resultados em uma **matriz de confusão**. Precisamos de todas as classes gramaticais que aparecem em `gold_standard` e `predictions`. Coletamos as categorias únicas com a função `set()` — um **conjunto** (*set*) é uma estrutura do Python com itens únicos, útil aqui para remover duplicatas:

```
# Coleta as classes gramaticais únicas em um conjunto, combinando as duas listas
pos_tags = set(gold_standard + predictions)

# Ordena o conjunto alfabeticamente e converte o resultado em lista
pos_tags = list(sorted(pos_tags))

# Imprime a lista resultante
pos_tags
```

```
['ADJ', 'AUX', 'DET', 'NOUN', 'PRON', 'PROPN', 'VERB']
```

Usamos essas categorias para montar uma tabela em que as **linhas** representam o padrão-ouro e as **colunas** as previsões do modelo. Percorremos cada par de itens (padrão-ouro, previsão) e somamos **+1** na célula correspondente. Por exemplo, o primeiro item de `gold_standard` é `ADJ` e o primeiro de `predictions` é `NOUN`:

```
# Imprime o primeiro item de cada lista
gold_standard[0], predictions[0]
```

```
('ADJ', 'NOUN')
```

Encontramos a linha `ADJ` e a coluna `NOUN` e somamos 1 àquela célula. A tabela completa (a **matriz de confusão**) fica assim:

	ADJ	AUX	DET	NOUN	PRON	PROPN	VERB
ADJ	2	0	0	1	0	0	0
AUX	0	2	0	0	0	0	0
DET	0	0	1	0	0	0	0
NOUN	0	0	0	1	0	0	1
PRON	0	0	0	0	0	1	0
PROPN	0	0	0	0	0	0	0
VERB	0	0	0	0	0	0	1

As previsões corretas formam uma **linha aproximadamente diagonal** na tabela. A partir dela, derivamos duas métricas por classe: **precisão** (*precision*) e **revocação** (*recall*).

- **Precisão** é a proporção de previsões corretas **por classe** — quantas das previsões daquela classe estavam certas. Por exemplo, a coluna `VERB` soma **2**, mas só **1** previsão está correta (na linha `VERB`); logo, a precisão de `VERB` é $1/2 = 0,5$. O mesmo vale para `NOUN`.
- **Revocação** é a proporção de previsões corretas dentre **todos os exemplos reais** daquela classe — quantas instâncias reais o modelo conseguiu “encontrar”. Por exemplo, a linha `ADJ` soma **3** (há três adjetivos no padrão-ouro), mas só **2** estão na coluna `ADJ`; logo, a revocação de `ADJ` é $2/3 \approx 0,66$. Para `NOUN`, a revocação é $1/2 = 0,5$.

A scikit-learn gera matrizes de confusão automaticamente com `confusion_matrix()`:

```
# Calcula a matriz de confusão para as duas listas e imprime o resultado
print(metrics.confusion_matrix(gold_standard, predictions))
```

```
[[2 0 0 1 0 0 0]
 [0 2 0 0 0 0 0]
 [0 0 1 0 0 0 0]
 [0 0 0 1 0 0 1]
 [0 0 0 0 0 1 0]
 [0 0 0 0 0 0 0]
 [0 0 0 0 0 0 1]]
```

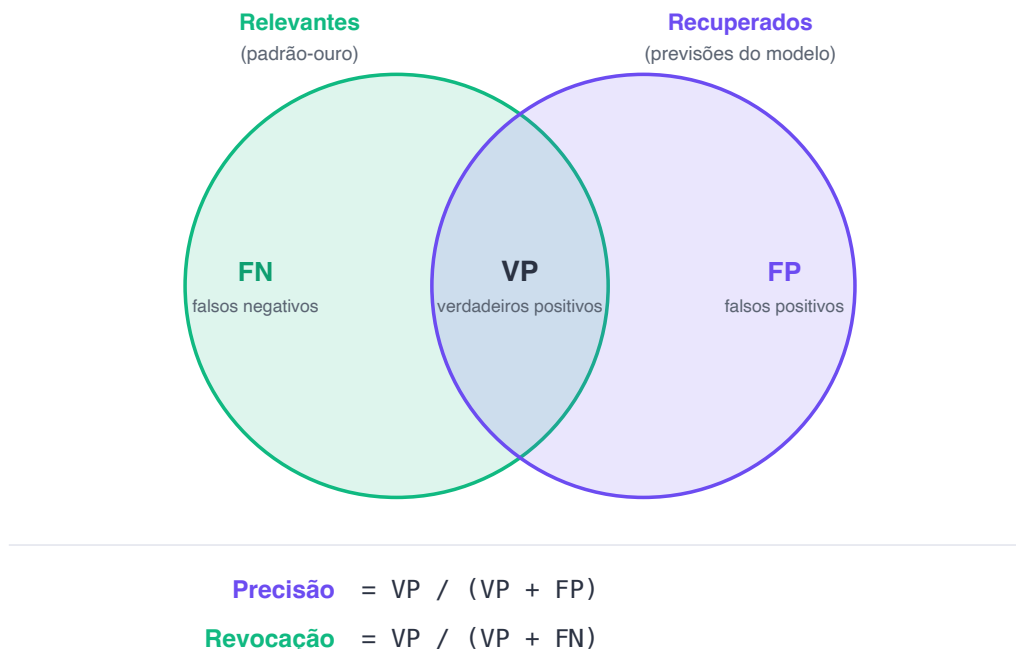


Figura 1: Precisão e revocação: os **relevantes** são o padrão-ouro; os **recuperados** são as previsões do modelo. A **precisão** olha para os recuperados (quantos acertou); a **revocação** olha para os relevantes (quantos encontrou).

4.3 Precisão e revocação com a scikit-learn

A **precisão** é implementada na função `precision_score()`. Como temos **mais de duas** classes, precisamos definir como os resultados de cada classe são combinados, pelo argumento `average`. Com `average=None`, a função calcula a precisão **de cada classe**. Também definimos `zero_division=0` para evitar erro quando uma classe de `predictions` não existe em `gold_standard` (nesses casos, a precisão é 0):

```
# Calcula a precisão entre as duas listas, para cada classe (classe gramatical)
precision = metrics.precision_score(gold_standard, predictions, average=None,
    ↪ zero_division=0)
```

```
# Chama a variável para examinar o resultado
precision
```

```
array([1. , 1. , 1. , 0.5, 0. , 0. , 0.5])
```

A saída é um **array NumPy**. Para combinar os rótulos de `pos_tags` com as pontuações de `precision`, usamos a função `zip()`, que une listas/arrays do mesmo tamanho; para visualizar, convertamos com `dict()`:

```
# Combina o conjunto 'pos_tags' com o array 'precision' usando zip();
# converte o resultado em um dicionário
dict(zip(pos_tags, precision))
```

```
{'ADJ': 1.0, 'AUX': 1.0, 'DET': 1.0, 'NOUN': 0.5, 'PRON': 0.0, 'PROPN': 0.0, 'VERB': 0.5}
```

Para uma **única** pontuação de precisão para todas as classes, usamos `average='macro'`, que trata cada classe como igualmente importante, independentemente de quantas instâncias ela tem:

```
# Calcula a precisão entre as duas listas e tira a média (macro)
macro_precision = metrics.precision_score(gold_standard, predictions, average='macro',
→ zero_division=0)

# Chama a variável para examinar o resultado
macro_precision
```

```
0.5714285714285714
```

A precisão **macro** é a soma das precisões dividida pelo número de classes — dá para verificar manualmente:

```
# Calcula a média macro manualmente: soma as precisões e divide
# pelo número de classes em 'precision'
sum(precision) / len(precision)
```

```
0.5714285714285714
```

A **revocação** é calculada do mesmo modo, com `recall_score()`:

```
# Calcula a revocacao entre as duas listas, para cada classe
recall = metrics.recall_score(gold_standard, predictions, average=None, zero_division=0)

# Combina 'pos_tags' com o array 'recall' e converte em dicionário
dict(zip(pos_tags, recall))
```

```
{'ADJ': 0.6666666666666666, 'AUX': 1.0, 'DET': 1.0, 'NOUN': 0.5, 'PRON': 0.0, 'PROPN':
→ 0.0, 'VERB': 1.0}
```

4.4 Relatório de classificação

A scikit-learn oferece a função `classification_report()`, que dá precisão e revocação de cada classe, junto com o **F1-score** — uma média equilibrada entre precisão e revocação (ambas contribuem igualmente), variando de 0 a 1:

```
# Imprime um relatorio de classificacao
print(metrics.classification_report(gold_standard, predictions, zero_division=0))
```

	precision	recall	f1-score	support
ADJ	1.00	0.67	0.80	3
AUX	1.00	1.00	1.00	2
DET	1.00	1.00	1.00	1
NOUN	0.50	0.50	0.50	2
PRON	0.00	0.00	0.00	1
PROPN	0.00	0.00	0.00	0
VERB	0.50	1.00	0.67	1
accuracy			0.70	10
macro avg	0.57	0.60	0.57	10

weighted avg	0.75	0.70	0.71	10
--------------	------	------	------	----

As pontuações da linha **macro avg** correspondem às que calculamos acima. A linha **weighted avg** (média ponderada) leva em conta o número de instâncias de cada classe; a coluna **support** conta quantas instâncias foram observadas em cada classe.

Quiz

Marque a alternativa correta (a resposta certa está destacada com ✓).

1. Um padrão-ouro (*gold standard*) é:

1. Dados verificados por humanos, usados como referência ✓
2. A saída bruta do modelo
3. Um dicionário do Python

2. Por que a concordância percentual é uma medida fraca?

1. Não distingue acerto real de acerto por acaso ✓
2. É difícil de calcular
3. Só funciona com duas classes

3. O kappa de Cohen mede a concordância entre quantos anotadores?

1. Dois ✓
2. Três
3. Qualquer número

4. A precisão de uma classe responde: das vezes em que o modelo *previu* essa classe,

1. quantas estavam certas ✓
2. quantas existiam no total
3. quantas ele deixou de encontrar

5. A revocação de uma classe responde: dos exemplos que *realmente* são dessa classe,

1. quantos o modelo encontrou ✓
2. quantos ele previu a mais
3. quantos estavam errados

6. O F1-score é:

1. A média equilibrada entre precisão e revocação ✓
2. A soma da precisão com a revocação
3. O maior valor entre os dois

Resumo da unidade

Nesta unidade, você aprendeu a:

1. entender o que é um **padrão-ouro** e por que ele é uma **abstração** do uso da língua;
2. medir a **confiabilidade** entre anotadores: concordância percentual, probabilidades por categoria e concordância esperada por acaso;

3. calcular e interpretar o **kappa de Cohen** (manualmente e com `cohen_kappa_score()`), usando os *benchmarks* de Landis & Koch;
4. avaliar o desempenho de modelos com **acurácia, matriz de confusão, precisão, revocação e F1-score**, usando a `scikit-learn` (`accuracy_score`, `confusion_matrix`, `precision_score`, `recall_score`, `classification_report`).

Exercícios sugeridos

1. Anote você mesmo os 10 *tweets* e peça a um colega que faça o mesmo; calcule a concordância percentual e o κ de Cohen com `cohen_kappa_score()`.
 2. Monte suas próprias listas `gold_standard` e `predictions` (com pelo menos 8 itens) e gere a matriz de confusão com `confusion_matrix()`.
 3. A partir do relatório de classificação, explique por que a classe `PROPN` tem `support` igual a 0 e o que isso significa.
 4. Calcule o F1-score de `VERB` **manualmente** a partir da precisão e da revocação (fórmula: $F1 = 2 \cdot (P \cdot R) / (P + R)$) e compare com o relatório.
-

Referências

- Rognvaldsson, E. et al. *Applied Language Technology* (MOOC). Universidade de Helsinque. <https://applied-language-technology.mooc.fi/>
- Landis, J. R.; Koch, G. G. (1977). The measurement of observer agreement for categorical data. *Biometrics*.
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR*. <https://scikit-learn.org/>

Gerenciando dados textuais com pandas

Trabalhando com Texto em Python I · Unidade 5

CiberExt 26-29 · FEELT38103 · Universidade Federal de Uberlândia

2026

Sumário

1. Importando dados para o pandas	1
2. Examinando DataFrames	4
3. Estendendo DataFrames	6
4. Salvando DataFrames	9
Quiz	10
Resumo da unidade	11

Objetivos de aprendizagem

Ao final desta unidade, você deverá saber:

- **importar dados** para um `DataFrame` do pandas;
- **explorar** dados armazenados em um `DataFrame`;
- **acrescentar** (estender) dados a um `DataFrame`;
- **salvar** os dados de um `DataFrame`.

O **pandas** é uma biblioteca do Python para trabalhar com **dados tabulares**. Começamos importando-a — usamos `as` para controlar o nome do módulo importado; por convenção, o pandas é abreviado como `pd`:

```
import pandas as pd
```

1. Importando dados para o pandas

1.1 A partir de um único arquivo

Formatos típicos para distribuir corpora incluem arquivos **CSV** (*Comma-separated Values*, valores separados por vírgula), **JSON** (*JavaScript Object Notation*) e texto simples. O pandas oferece muitas funções para ler dados em vários formatos — dá até para importar planilhas do Excel!

O exemplo a seguir carrega um recorte do *SFU Opinion and Comments Corpus* (SOCC), que reúne artigos de opinião do jornal canadense *The Globe and Mail*. Usamos a função `read_csv()`, que recebe uma *string* com o **caminho** do arquivo:

```
# Lê o arquivo CSV e atribui a saída a variável 'socc'
socc = pd.read_csv('data/socc_gnm_articles.csv')
```

```
# Examina o tipo do objeto guardado em 'socc'
type(socc)
```

```
pandas.core.frame.DataFrame
```

O pandas faz todo o trabalho pesado e devolve o conteúdo do CSV em um **DataFrame**, a estrutura de dados nativa do pandas. Usamos o método `head()` para ver as primeiras cinco linhas:

```
# Imprime as primeiras cinco linhas do DataFrame
socc.head(5)
```

O **DataFrame** tem forma **tabular**, com colunas como `article_id`, `title`, `author`, `article_text` etc., além de um **índice** para cada linha (0, 1, 2, ...). Uma versão compacta (algumas colunas omitidas para caber):

índice	article_id	title	author	ntop_level_comments
0	26842506	The Tories deserve another mandate...	GLOBE EDITORIAL	1378.0
1	26055892	Harper hysteria a sign of closed liberal minds	Konrad Yakabuski	455.0
2	6929035	Too many first nations people live in a dream...	Jeffrey Simpson	433.0
3	19047636	The Globe's editorial board endorses Tim Hudak...	GLOBE EDITORIAL	432.0
4	11672346	Disgruntled Arab states look to strip Canada...	Campbell Clark	411.0

O acessador `.at[]` permite inspecionar **um único item**. Vamos ver o valor da coluna `title` no índice 123:

```
socc.at[123, 'title']
```

```
"How Toronto got a 'world-class,' gold-plated, half-billion-dollar empty train"
```

1.2 A partir de múltiplos arquivos

Outro cenário comum é ter **vários arquivos** de texto para carregar. Primeiro, coletamos os arquivos com a classe **Path** (vista na Unidade 1):

```
# Importa a classe Path
from pathlib import Path

# Cria um objeto Path que aponta para o diretório com os dados
corpus_dir = Path('data')
```

```
# Coleta todos os arquivos .txt no diretório do corpus
corpus_files = list(corpus_dir.glob('*.txt'))

# Verifica os arquivos do corpus
corpus_files
```

```
[PosixPath('data/WP_1990-08-10-25A.txt'),
 PosixPath('data/NYT_1991-01-16-A15.txt'),
 PosixPath('data/WP_1991-01-17-A1B.txt')]
```

Para acomodar os dados, criamos um **DataFrame** **vazio** e definimos sua forma de antemão: o número de linhas (**index**) e os nomes das colunas (**columns**). Determinamos o número de linhas com a função **range()**, entre 0 e a quantidade de arquivos (obtida com **len()**). Para as colunas, passamos uma lista de *strings* ao argumento **columns**:

```
# Cria um DataFrame e atribui o resultado a variável 'df'
df = pd.DataFrame(index=range(0, len(corpus_files)), columns=['filename', 'text'])

# Chama a variável para inspecionar a saída
df
```

índice	filename	text
0	NaN	NaN
1	NaN	NaN
2	NaN	NaN

Agora percorremos os objetos **Path** em **corpus_files**, lemos o conteúdo e o adicionamos ao **DataFrame** com o acessador **.at**:

```
# Percorre os arquivos do corpus e conta cada volta com enumerate()
for i, f in enumerate(corpus_files):

    # Lê o conteúdo do arquivo
    text = f.read_text(encoding='utf-8')

    # Pega o nome do arquivo do objeto Path
    filename = f.name

    # Atribui o texto do arquivo ao índice 'i' na coluna 'text' usando
    # o acessador .at -- isso modifica o DataFrame "no lugar" (in place);
    # não é preciso atribuir o resultado a uma variável
    df.at[i, 'text'] = text

    # Faz o mesmo com o nome do arquivo
    df.at[i, 'filename'] = filename
```

O **DataFrame** fica populado com os nomes dos arquivos e o texto:

índice	filename	text
0	WP_1990-08-10-25A.txt	*We Don't Stand for Bullies': Diverse Voices...
1	NYT_1991-01-16-A15.txt	U.S. TAKING STEPS TO CURB TERRORISM: F.B.I. I...
2	WP_1991-01-17-A1B.txt	U.S., Allies Launch Massive Air War Against T...

2. Examinando DataFrames

Os `DataFrames` guardam muita informação, geralmente organizada em **colunas**, acessíveis pelo atributo `columns`:

```
# Recupera as colunas e seus nomes
socc.columns
```

```
Index(['article_id', 'title', 'article_url', 'author', 'published_date',
      'ncomments', 'ntop_level_comments', 'article_text'],
      dtype='object')
```

Uma coluna inteira é acessada com colchetes `[]`, colocando o nome da coluna como *string* — como as chaves de um dicionário. Vamos recuperar a coluna `author`:

```
# Recupera o conteúdo da coluna 'author' no DataFrame 'socc'
socc['author']
```

```
0      GLOBE EDITORIAL
1      Konrad Yakabuski
2      Jeffrey Simpson
...
10337    Adam Radwanski
10338    GLOBE EDITORIAL
Name: author, Length: 10339, dtype: object
```

A coluna contém 10 339 objetos (veja `Length` e `dtype`); os números à esquerda são o índice. Cada coluna de um `DataFrame` é, na verdade, um objeto `Series` — pense no `DataFrame` como a tabela inteira, cujas colunas são `Series`:

```
# Verifica o tipo de 'socc' e de 'socc['author']'
type(socc), type(socc['author'])
```

```
(pandas.core.frame.DataFrame, pandas.core.series.Series)
```

Nota: ao imprimir um `DataFrame` ou `Series`, o pandas **omite** tudo entre as cinco primeiras e as cinco últimas linhas por padrão — conveniente ao trabalhar com milhares de linhas.

O método `value_counts()` conta os valores únicos de uma `Series`:

uma coluna = Series

índice	filename	ncomments	author
0	WP_1990.txt	2187.0	GLOBE ED.
1	NYT_1991.txt	1103.0	K. Yakabuski
2	WP_1991.txt	1164.0	J. Simpson

linha
(um registro)

A tabela inteira é um **DataFrame**; cada coluna é uma **Series**.

Figura 1: Anatomia de um **DataFrame**: a tabela inteira é um **DataFrame**, cada **coluna** é uma **Series**, cada **linha** é um registro e a coluna cinza à esquerda é o **índice**.

```
# Conta os valores únicos na coluna 'author'
socc['author'].value_counts()
```

```
GLOBE EDITORIAL          2712
Jeffrey Simpson          649
Margaret Wente           547
...
Jessica Scott-Reid        1
Kenneth Oppel             1
Name: author, Length: 1896, dtype: int64
```

Sem surpresa, a equipe editorial do *The Globe and Mail* assina a maioria dos editoriais! Podemos **visualizar** o resultado chamando o método `.plot()`, que usa a biblioteca **matplotlib**. Passamos a *string* `bar` ao argumento `kind` para um gráfico de barras, e `[:10]` para os dez autores mais prolíficos:

```
# Magia do Jupyter que permite renderizar graficos do matplotlib no notebook!
# Voce so precisa executar este comando uma vez.
%matplotlib inline

# Conta os valores da coluna 'author' e limita ao top-10 antes de plotar.
socc['author'].value_counts()[:10].plot(kind='bar')
```

Dica: o `.plot()` gera uma **imagem** (gráfico de barras) exibida no notebook; a saída de texto é apenas `<AxesSubplot: >`. Para outros tipos de gráfico, mude o argumento `kind` (ex.: `'line'`, `'hist'`, `'pie'`).

Para colunas **numéricas**, o método `describe()` dá estatísticas descritivas básicas:

```
# Estatísticas descritivas básicas da coluna 'ntop_level_comments'
socc['ntop_level_comments'].describe()
```

```
count    10339.000000
mean      26.384273
std       39.786923
min        0.000000
25%        1.000000
50%       14.000000
75%       35.000000
max      1378.000000
Name: ntop_level_comments, dtype: float64
```

Lendo a saída: há 10 339 linhas; a média (**mean**) de comentários por editorial é ≈ 26 , mas com forte variação (desvio-padrão **std** de quase 40). O mínimo (**min**) é 0 (alguns editoriais não têm comentários). O primeiro quartil (**25%**) mostra que 25% dos dados têm 1 comentário ou menos; a mediana (**50%**) é 14; o terceiro quartil (**75%**) é 35; e o mais comentado (**max**) tem 1 378 comentários.

2.1 Filtrando linhas com `.loc`

Como encontrar os artigos com **zero** comentários? Usamos o acessador `.loc` para selecionar linhas com base em seus valores. Como `=` é reservado para atribuição, a comparação “é igual a” usa **dois** sinais (`==`):

```
# Pega as linhas sem comentários de nível superior
socc.loc[socc['ntop_level_comments'] == 0]
```

```
[... 2542 rows x 8 columns ...]
```

Isso retorna 2 542 linhas em que `ntop_level_comments` é zero. Para visões mais complexas, combinamos critérios com o operador `&` (“E” lógico) — cada critério deve ficar entre **parênteses** `()`. Vamos verificar se o primeiro autor do resultado (Hayden King) escreveu outros artigos com zero comentários:

```
# Número de comentários de nível superior para o autor Hayden King
socc.loc[(socc['ntop_level_comments'] == 0) & (socc['author'] == 'Hayden King')]
```

Isso retorna apenas a linha de índice 7797 — o único artigo dele sem comentários.

3. Estendendo DataFrames

É fácil **acrescentar** informação a um `DataFrame`. Um cenário comum: carregar dados de um arquivo, fazer análises e guardar os resultados no mesmo `DataFrame`. Adicionamos uma coluna vazia com o acessador de coluna `[]` e o tipo `None`:

```
# Adiciona uma nova coluna chamada 'comments_ratio' ao DataFrame
socc['comments_ratio'] = None
```

Vamos **preencher** a coluna calculando a proporção de comentários de nível superior (comentários sobre o artigo) em relação a todos os comentários — basta dividir uma coluna pela outra:

```
# Preenche a coluna 'comments_ratio' com a razão entre comentários de
# nível superior e o total de comentários
```

```
socc['comments_ratio'] = socc['ntop_level_comments'] / socc['ncomments']
```

Os acessadores de coluna podem ser usados de forma muito flexível para acessar e manipular dados, como nesta divisão. Uma amostra do resultado:

índice	ntop_level_comments	ncomments	comments_ratio
0	1378.0	2187.0	0.630087
1	455.0	1103.0	0.412511
2	433.0	1164.0	0.371993

Mas alguns artigos **não** receberam comentários — nesses casos, teríamos dividido zero por zero:

```
# Imprime os cinco primeiros artigos sem comentários de nível superior
socc.loc[socc['ntop_level_comments'] == 0].head(5)
```

Para essas linhas, `comments_ratio` fica marcada como `NaN` (*not a number*, “não é um número”): a divisão foi feita, mas o resultado não é um número. O pandas **ignora automaticamente** os `NaN` nos cálculos, como mostra o `describe()`:

```
# Estatísticas descritivas da coluna 'comments_ratio'
socc['comments_ratio'].describe()
```

```
count    7797.000000
mean      0.537057
std       0.205398
min       0.083333
25%      0.384615
50%      0.485714
75%      0.647059
max       1.000000
Name: comments_ratio, dtype: float64
```

Note a diferença no `count`: só 7 797 itens (dos 10 339) entraram no cálculo — os `NaN` foram ignorados.

3.1 Aplicando processamento de linguagem a uma coluna

E se quiséssemos fazer PLN e guardar os resultados no `DataFrame`? Vamos selecionar artigos no primeiro quartil de `comments_ratio` e com mais de 200 comentários (`ncomments`), usando `&`:

```
# Filtra o DataFrame por artigos muito comentados e atribui o resultado a 'talk'
talk = socc.loc[(socc['comments_ratio'] <= 0.384) & (socc['ncomments'] >= 200)]
```

Importamos o `spaCy` e carregamos um modelo **médio** para o inglês:

```
# Importa a biblioteca spaCy
import spacy

# Note que agora carregamos um modelo de tamanho medio!
nlp = spacy.load('en_core_web_md')
```

Ao criar uma coluna nova em `talk`, o pandas emite um `SettingWithCopyWarning`, porque `talk` é apenas um recorte (*slice*) do `DataFrame` original. Atribuir uma coluna a apenas uma parte quebraria a estrutura

tabular. A solução é criar uma **cópia profunda** (*deep copy*) do recorte com o método `.copy()`:

```
# Cria uma copia profunda do DataFrame
talk = talk.copy()

# Cria uma nova coluna chamada 'processed_title'
talk['processed_title'] = None
```

Para processar os títulos, usamos o método `apply()` de um `DataFrame`, que **aplica** o que recebe a cada linha da coluna. Passamos o modelo `nlp` — ou seja, aplicamos o modelo aos títulos (que são *strings*) da coluna `title`:

```
# Aplica o modelo de linguagem 'nlp' ao conteúdo da coluna 'title'
talk['processed_title'] = talk['title'].apply(nlp)
```

Cada célula da coluna `processed_title` agora contém um objeto `Doc` do `spaCy`:

```
# Pega o valor da coluna 'processed_title' na linha de índice 2
talk.at[2, 'processed_title']

# Verifica o tipo do objeto contido
type(talk.at[2, 'processed_title'])
```

```
Too many first nations people live in a dream palace
spacy.tokens.doc.Doc
```

3.2 Definindo a própria função e aplicando com `apply()`

Vamos definir nossa própria função para buscar os **lemas de cada substantivo** do título. Funções em Python são definidas com `def`, seguido do nome e dos parâmetros entre parênteses:

```
# Define uma função 'get_nouns' que recebe um único objeto como entrada.
# Referimo-nos a essa entrada pela variável 'nlp_text'.
def get_nouns(nlp_text):

    # Primeiro garantimos que a entrada é do tipo correto,
    # usando 'assert' para checar o tipo
    assert type(nlp_text) == spacy.tokens.doc.Doc

    # Prepara uma lista vazia para os lemas
    lemmas = []

    # Percorre o objeto Doc
    for token in nlp_text:

        # Se a classe gramatical detalhada do token for substantivo (NN)
        if token.tag_ == 'NN':

            # Acrescenta o lema do token a lista de lemas
            lemmas.append(token.lemma_)

    # Ao fim do laço, retorna a lista de lemas
    return lemmas
```

Aplicamos a função com `apply()`, criando a coluna `nouns` automaticamente pela atribuição:


```
# Aplica a função 'get_nouns' a coluna 'processed_title'
talk['nouns'] = talk['processed_title'].apply(get_nouns)
```

Uma amostra dos títulos processados e seus substantivos:

índice	title	nouns
2	Too many first nations people live in a dream...	[dream, palace]
5	Fifty years in Canada, and now I feel like a s...	[class, citizen]
8	A nation of \$100,000 firefighters	[nation]

Nota: não é necessário criar uma coluna vazia antes — o pandas cria a coluna nova automaticamente na atribuição, como em `talk['nouns']`.

3.3 Extraíndo dados para estruturas nativas do Python

O método `tolist()` extrai o conteúdo de uma `Series` para uma **lista**:

```
# Converte a pandas Series em uma lista
noun_list = talk['nouns'].tolist()
```

```
# Mostra os dez primeiros
noun_list[:10]
```

```
[['dream', 'palace'], ['agency'], ['class', 'citizen'], ['right'],
 ['nation'], [], ['reform'], ['leader', 'parade'], ['pm'],
 ['government', 'monopoly']]
```

Temos uma **lista de listas** (cada linha da coluna `nouns` é uma lista). Vamos “achatar” tudo em uma única lista `final_list` com o método `extend()`:

```
# Prepara a lista vazia
final_list = []

# Percorre cada lista dentro da lista de listas
for nlist in noun_list:

    # Estende a lista final com a lista atual
    final_list.extend(nlist)

# Verifica o tamanho da lista
len(final_list)
```

884

Para plotar os 10 substantivos mais frequentes, convertamos `final_list` em uma `Series`, contamos com `value_counts()` e plotamos:

```
# Converte a lista em uma pandas Series, conta os substantivos únicos com
# value_counts(), pega os 10 mais frequentes [:10] e plota em barras.
pd.Series(final_list).value_counts()[:10].plot(kind='bar')
```

4. Salvando DataFrames

DataFrames podem ser salvos como objetos **serializados** (*pickled*) com o método `to_pickle()`, que recebe o caminho do arquivo. Vamos salvar o `df` com os três artigos:

```
# Grava o DataFrame em disco usando pickle
df.to_pickle('data/pickled_df.pkl')
```

Segurança (importante no CiberExt): o formato *pickle* serializa objetos Python arbitrários e, ao ser **carregado**, pode **executar código arbitrário**. **Nunca** faça `read_pickle()` (nem `pickle.load`, `joblib.load`, `numpy` com `allow_pickle=True` etc.) em arquivos de **fontes não confiáveis** — um `.pkl` malicioso compromete a máquina. Use *pickle* apenas com dados que **você mesmo** gerou; para intercâmbio entre pessoas/sistemas, prefira formatos que não executam código, como **CSV**, **JSON** ou **Parquet** (`to_csv/read_csv`, `to_json/read_json`, `to_parquet/read_parquet`).

Verificamos lendo de volta com `read_pickle()`:

```
# Lê o DataFrame serializado e atribui o resultado a 'df_2'
df_2 = pd.read_pickle('data/pickled_df.pkl')
```

Comparando os dois **DataFrames** com `==`, obtemos um valor booleano (**True/False**) para cada célula:

```
# Compara os DataFrames 'df' e 'df_2'
df == df_2
```

índice	filename	text
0	True	True
1	True	True
2	True	True

Tudo **True** — os dados foram salvos e recarregados com sucesso.

Quiz

Marque a alternativa correta (a resposta certa está destacada com ✓).

1. Por convenção, o **pandas** é importado com qual apelido?

1. `pd` ✓
2. `pandas`
3. `dt`

2. Cada coluna de um **DataFrame** é, na verdade, um objeto:

1. `Series` ✓
2. `DataFrame`
3. `list`

3. Um valor `NaN` indica:

1. Um valor ausente (“não é um número”) ✓
2. O número zero
3. Infinito

4. Qual operador combina dois critérios num filtro `.loc`?

1. `&` ✓
2. `and`
3. `+`

5. Por que criar uma cópia com `.copy()` de um recorte do `DataFrame`?

1. Para evitar o `SettingWithCopyWarning` e não alterar o original ✓
2. Para acelerar o cálculo
3. É obrigatório em toda operação

6. (Segurança) Por que *não* usar `read_pickle` em um arquivo de origem não confiável?

1. Porque pode executar código arbitrário na sua máquina ✓
2. Porque é mais lento que o CSV
3. Porque não consegue ler texto

Resumo da unidade

Nesta unidade, você aprendeu a:

1. **importar** dados para um `DataFrame` a partir de um único arquivo (`read_csv()`) ou de vários arquivos (`Path/glob` + laço com `.at`);
2. **examinar** `DataFrames`: `columns`, colunas como `Series`, `value_counts()`, `describe()`, `.plot()` e filtragem com `.loc` e critérios combinados (`&`, `==`);
3. **estender** `DataFrames`: adicionar colunas, calcular entre colunas, lidar com `NaN`, usar `.copy()` para evitar o `SettingWithCopyWarning`, e aplicar funções (do spaCy ou próprias) com `apply()`;
4. **extrair** dados com `tolist()/extend()` e **salvar/carregar** com `to_pickle()/read_pickle()`.

Exercícios sugeridos

1. Carregue os três arquivos de notícias da Unidade 1 em um `DataFrame` (colunas `filename` e `text`) e conte quantos caracteres cada texto tem, guardando o resultado em uma nova coluna `n_chars`.
2. No `DataFrame` `socc`, use `.loc` para selecionar os artigos do autor `GLOBE EDITORIAL` com mais de 500 comentários.
3. Escreva uma função `get_verbs()` (análoga a `get_nouns`) que colete os lemas dos **verbos** (`token.pos_ == 'VERB'`) e aplique-a a uma coluna de títulos processados.
4. Salve um `DataFrame` em CSV com `to_csv('saida.csv', index=False)` e recarregue-o com `read_csv()`, conferindo se as colunas foram preservadas.

Referências

- Rögnavaldsson, E. et al. *Applied Language Technology* (MOOC). Universidade de Helsinque. <https://applied-language-technology.mooc.fi/>
- Kolhatkar, V. et al. (2020). The SFU Opinion and Comments Corpus. *Corpus Pragmatics*.
- McKinney, W. et al. *pandas: powerful Python data analysis toolkit*. <https://pandas.pydata.org/>